GSAS @NIDA : Deep Learning

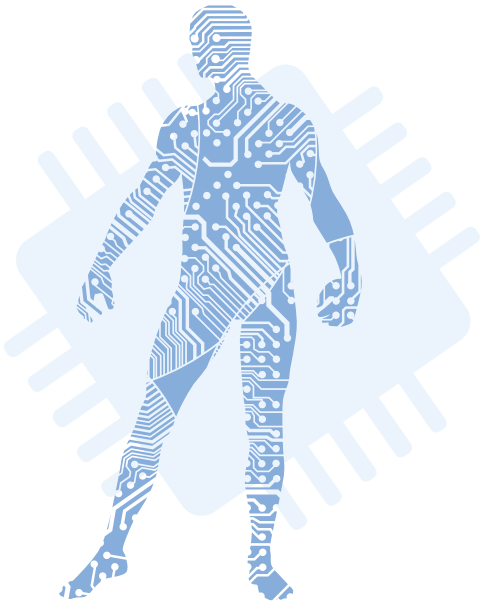
CNN 2

Assoc.Prof.Thitirat Siriborvornratanakul, Ph.D.

Email: thitirat@as.nida.ac.th
Website: http://as.nida.ac.th/~thitirat/

1

OUTLINE



01 State-of-the-art Classifiers
Dig in CNN's components through famous state-of-the-art CNN-based image classifiers

02 Foundation Models
Jump start our CNNs with the big foundation models and their pre-trained weights

03 Demystify CNNs
Make CNNs more transparent and explainable

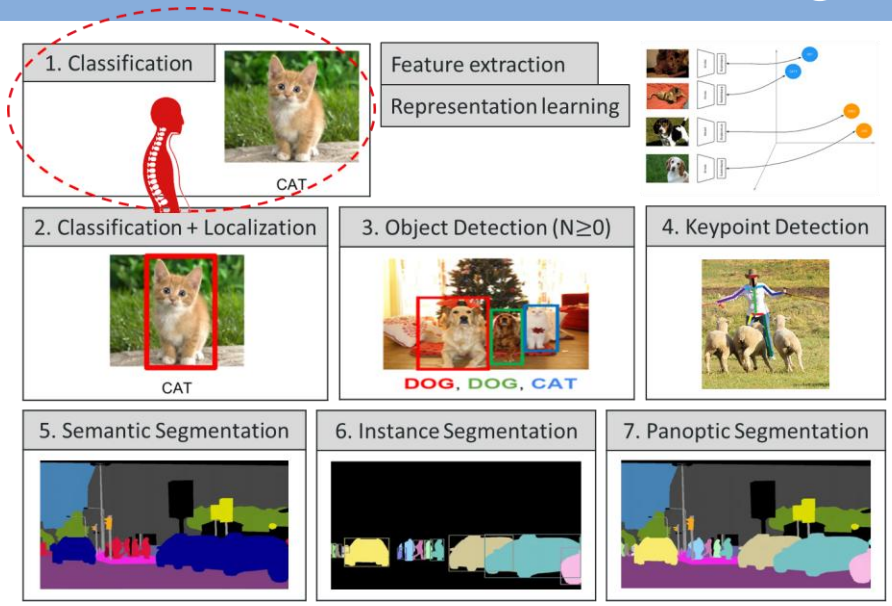
2

SOTA Classifiers

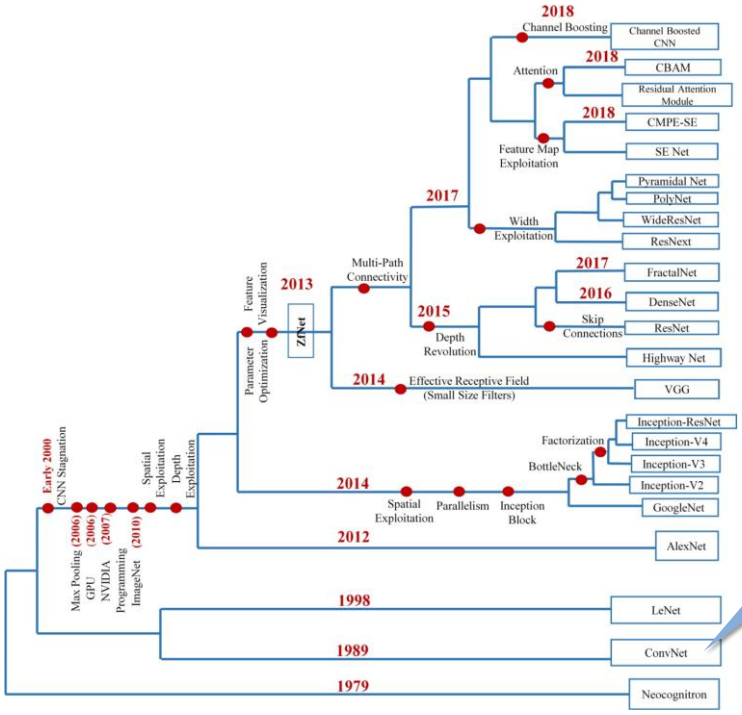
Dig in CNN's components through famous state-of-the-art CNN-based image classifiers

3

Basic DL tasks on 2D Image



4



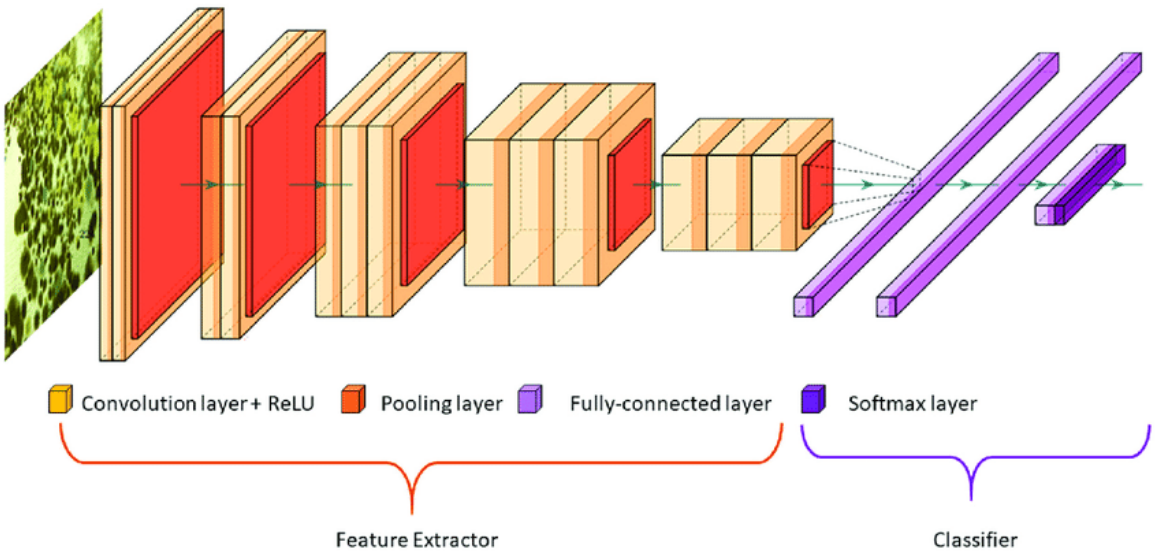
The first multilayered CNN named ConvNet whose origin rooted in Fukushima's Neocognitron

LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, **1**(4), pp. 541-551, 1989. http://yann.lecun.com/exdb/publis/pdf/lecun-89e.pdf

"A survey of the recent architectures of deep convolutional neural networks," *Artificial Intelligence Review*, **53**, pp. 5455-5516, 2020. https://link.springer.com/article/10.1007/s10462-020-09825-6

5

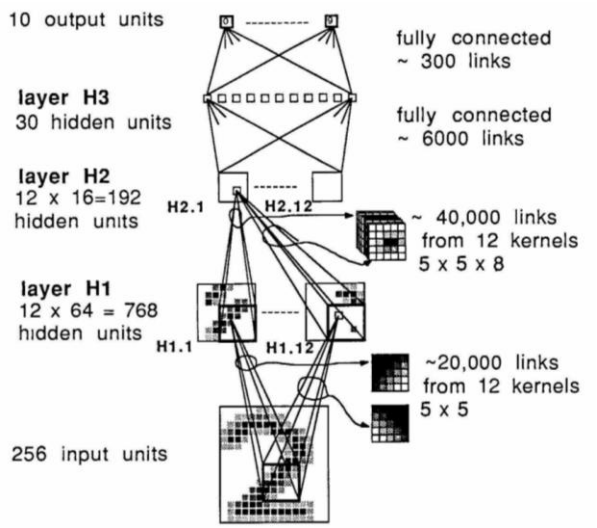
CNN for Image Classification



6

ConvNet (1989)

- The first multilayered CNN named **ConvNet** whose origin rooted in Fukushima's Neocognitron

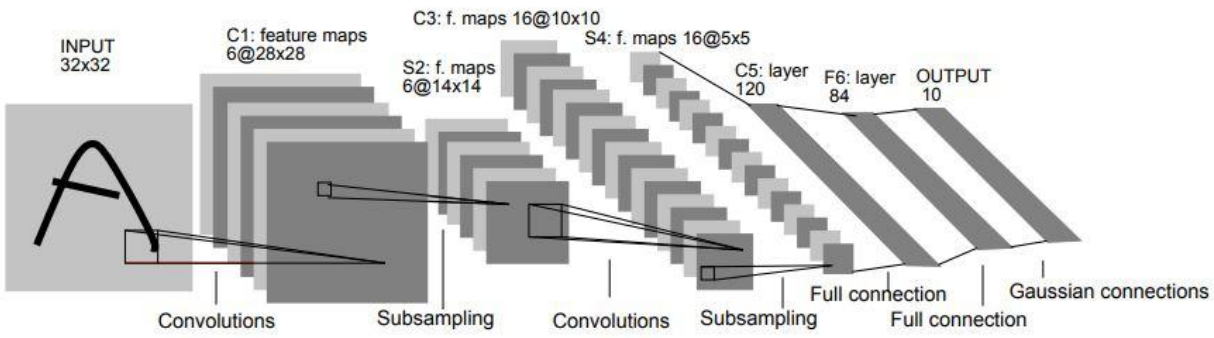


LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, **1**(4), pp. 541-551, 1989. http://yann.lecun.com/exdb/publis/pdf/lecun-89e.pdf

7

LeNet-5 (1998)

- **LeNet-5** for MNIST dataset by Yann LeCun et al.

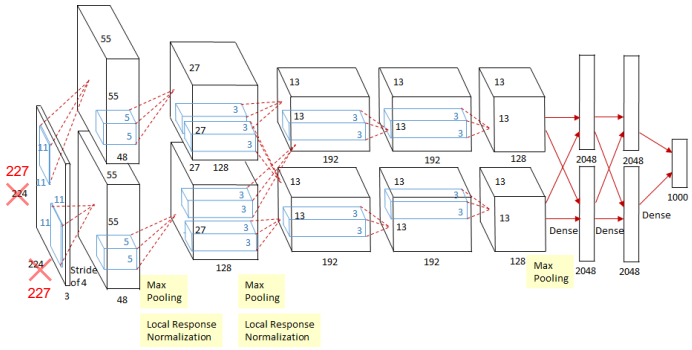


Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner, "Gradient-based learning applied to document recognition," In Procs. of the IEEE, 1998 http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf

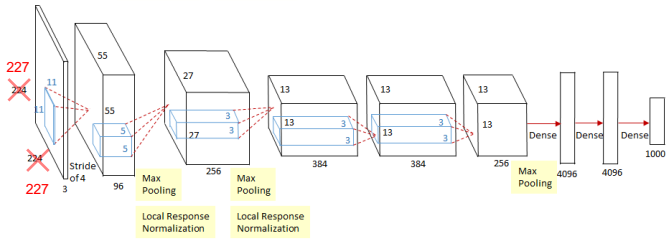
8

AlexNet (ImageNet 2012, NeurIPS 2012)

- **AlexNet** (2 GPUs): 8 layers
  - Winner of ImageNet 2012
  - Trained for 90 epochs in 6 days using two NVIDIA GeForce GTX 580 GPUs (each with 3GB memory)
  - Two regularization techniques: dropout and data augmentation



- **CaffeNet** (1 GPU):



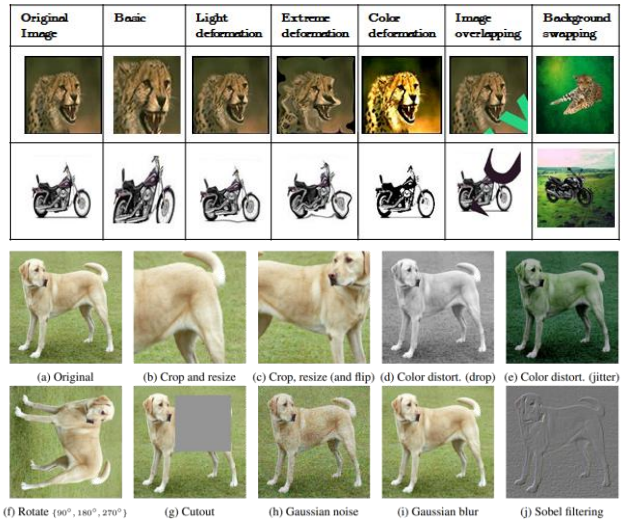
A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," In Procs. of NeurIPS, 2012 https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

9

10

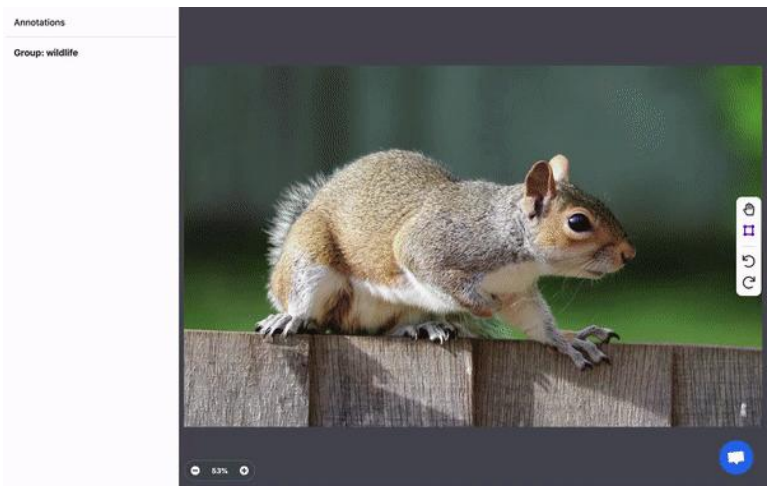
(2D Image) Data Augmentation

- Artificially increase the size of the training set by generating many realistic variants of each training instance
- **Data augmentation** reduces overfitting, making it a regularization technique.



11

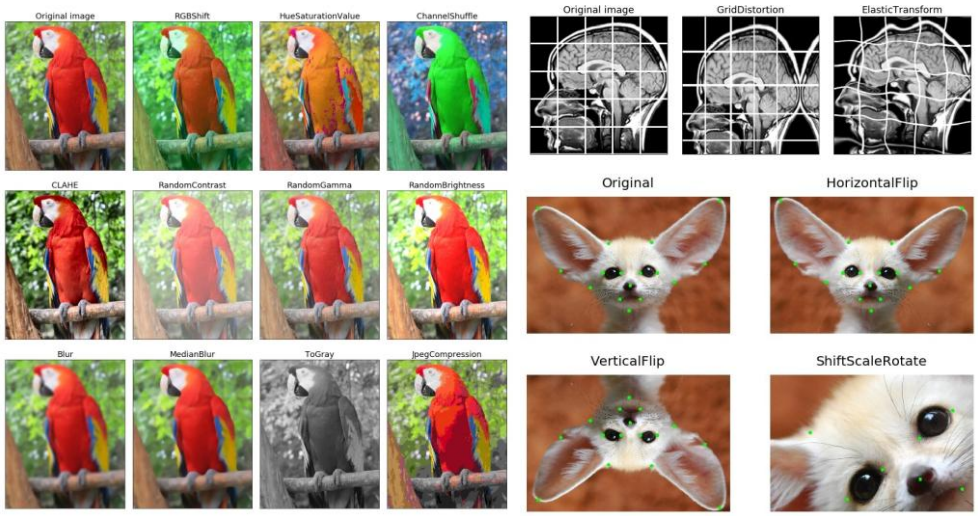
(2D Image) Data Augmentation



https://roboflow.com

12

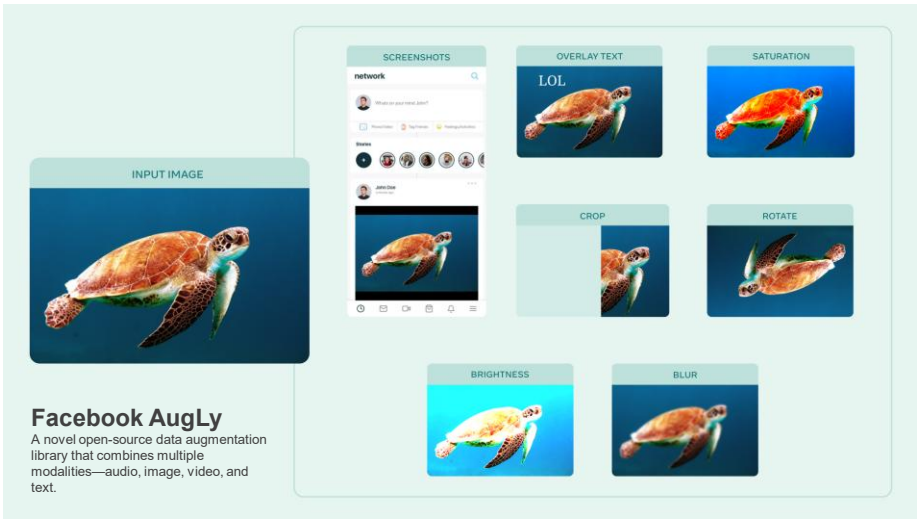
(2D Image) Data Augmentation



https://github.com/albumentations-team/albumentations

13

(2D Image) Data Augmentation



17JUN2021: https://ai.facebook.com/blog/augly-a-new-data-augmentation-library-to-help-build-more-robust-ai-models

14

(2D Image) Data Augmentation

Image augmentation layers

- AugMix layer → Mix many augmentations w/o degrading or drifting off the data manifold , ICLR 2020 https://arxiv.org/abs/1912.02781
- CutMix layer → Patches are cut and pasted among training images, ICCV 2019 https://arxiv.org/abs/1905.04899
- Equalization layer → Apply histogram equalization on image channels
- MixUp layer → MixUp augmentation technique for object detection, arXiv 2019 https://arxiv.org/abs/1902.04103
- Pipeline layer → Build a preprocessing pipeline for image data augmentation; the result is a plain layer, not a model.
- RandAugment layer → An all-in-one image augmentation layer with a reduced search space, NIPS 2020 https://arxiv.org/abs/1909.13719
- RandomBrightness layer
- RandomColorDegeneration layer → Randomly perform the color degeneration operation (make colors appear more dull)
- RandomColorJitter layer
- RandomContrast layer
- RandomCrop layer
- RandomElasticTransform layer → Apply random elastic transformations
- RandomErasing layer → Random patches of an image are erased
- RandomFlip layer
- RandomGaussianBlur layer
- RandomGrayscale layer → Random conversion of RGB images to grayscale
- RandomHue layer
- RandomInvert layer
- RandomPerspective layer → Distort the perspective of input images by shifting their corner points, simulating a 3D-like transformation
- RandomPosterization layer
- RandomRotation layer
- RandomSaturation layer
- RandomSharpness layer
- RandomShear layer
- RandomTranslation layer
- RandomZoom layer
- Solarization layer → Applies `(max_value - pixel + min_value)` for each pixel in the image

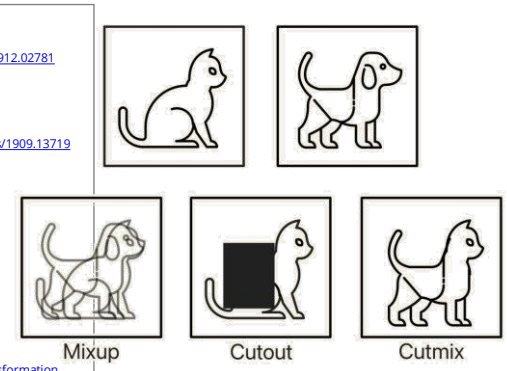


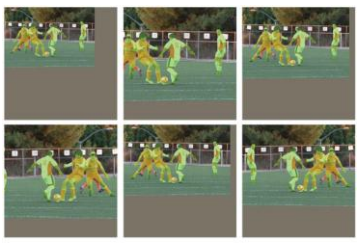
Image credit: 14APR2023 https://towardsdatascience.com/cutout-mixup-and-cutmix-implementing-modern-image-augmentations-in-pytorch-a9d7db3074ad/

https://keras.io/api/layers/preprocessing_layers/image_augmentation/

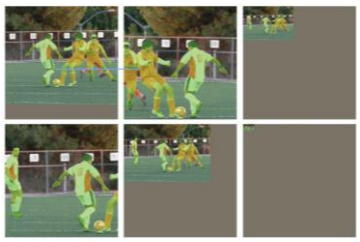
15

Scale Jittering Augmentation

- **Scale jittering augmentation:**
  Available in `torchvision.transforms.v2.ScaleJitter(…)`
  - Randomly resize and crop images
  - If images are made smaller than their original size, then the images are padded with gray pixel values.
  - **Standard scale jittering (SSJ):** a resize range of 0.8 to 1.25 of the original image size
  - **Large-scale jittering (LSJ):** a resize range of 0.1 to 2.0 of the original image size
- **Research finding:**
  - LSJ yields significant performance improvements over the standard scale jittering used in most prior works.



(a) **Standard Scale Jittering (SSJ)**



(b) **Large Scale Jittering (LSJ)**

"Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation," CVPR 2021 (Oral) https://arxiv.org/abs/2012.07177, https://github.com/conradry/copy-paste-aug

16

Copy-Paste Augmentation (for Instance Segmentation)

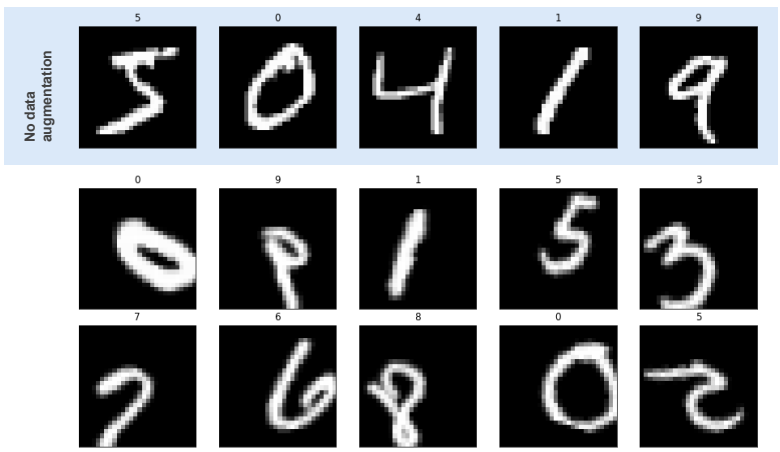
- **Copy-paste augmentation:** A simple strategy of randomly picking diverse objects of various scales and pasting them to new background images at random locations
- **Research finding:** This simple mechanism of pasting objects randomly is good enough and can provide solid gains on top of strong baselines.

"Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation," CVPR 2021 (Oral) https://arxiv.org/abs/2012.07177, https://github.com/conradry/copy-paste-aug

17

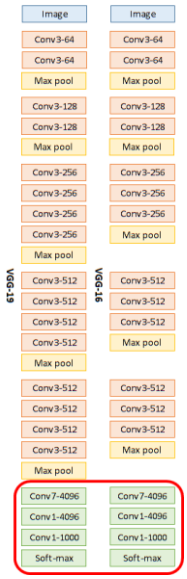
EX1: (2D Image) Data augmentation

- Example of training a CNN model with image data augmentation
- This example is based on the MNIST dataset and Keras's built-in image augmentation commands.

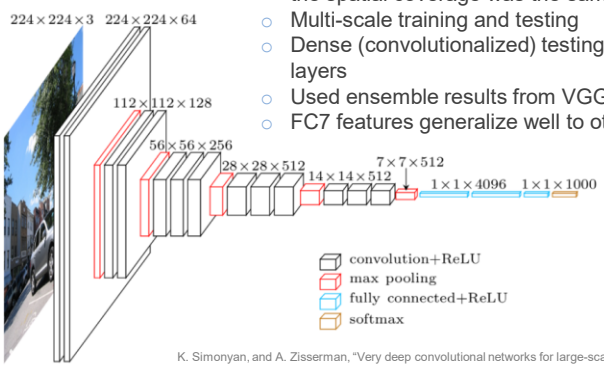


18

Oxford VGG (ImageNet 2014, ICLR 2015)



- **VGG16** (16 layers) and **VGG19** (19 layers): The 1st runner up in ImageNet 2014
- **VGG16** is one of the most popular CNN backends. Although VGG19 is a bit better in accuracy, it also consumes more memory.
- Trained 2-3 weeks using four NVIDIA Titan Black (6GB) GPUs



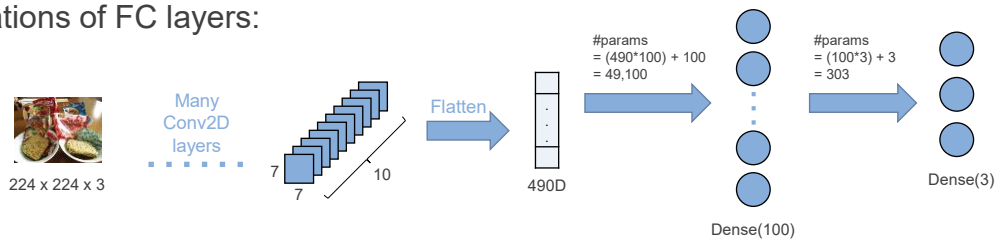
- **Tricks:**
  - Used a hierarchy of 3x3 convolutions instead of 5x5 and 11x11 convolutions; the spatial coverage was the same but the number of parameters was reduced
  - Multi-scale training and testing
  - Dense (convolutionalized) testing: during testing, replace FC layers with CNN layers
  - Used ensemble results from VGG16 and VGG19 for the best results
  - FC7 features generalize well to other tasks, suitable for transfer learning

K. Simonyan, and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," In Procs. of ICLR, 2015 https://arxiv.org/abs/1409.1556

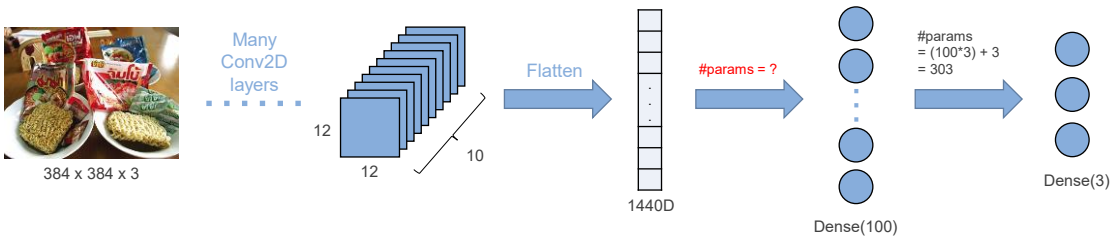
19

FC layer ⇔ Conv layer

- Limitations of FC layers:



  - Using the above network, what if the size of input image is increased to 384x384x3 ?

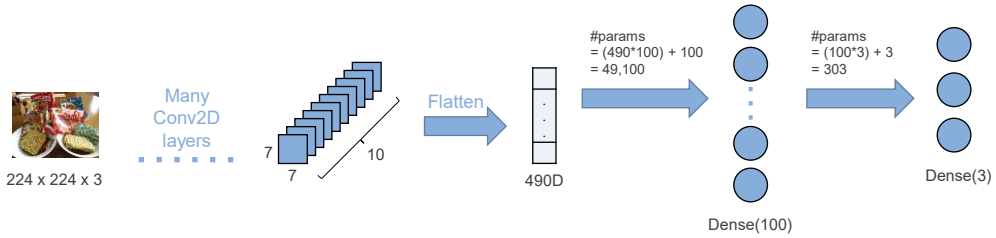


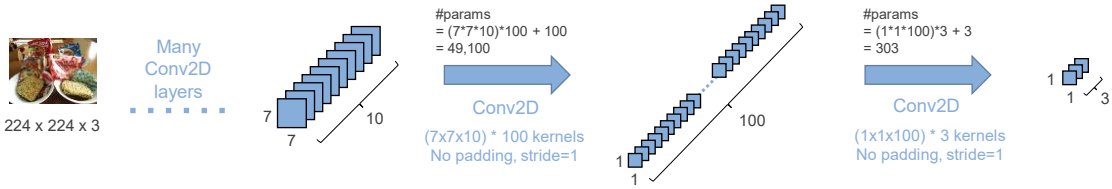
Further reading: http://cs231n.github.io/convolutional-networks/#convert

20

FC layer ⇔ Conv layer

- Deconvolutionalize: convert from a Conv layer to an equivalent FC layer
- Convolutionalize: convert from a FC layer to an equivalent Conv layer

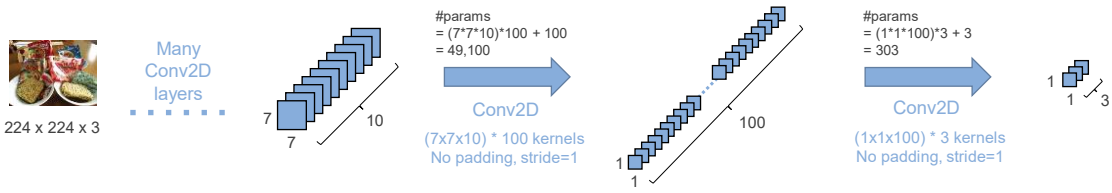


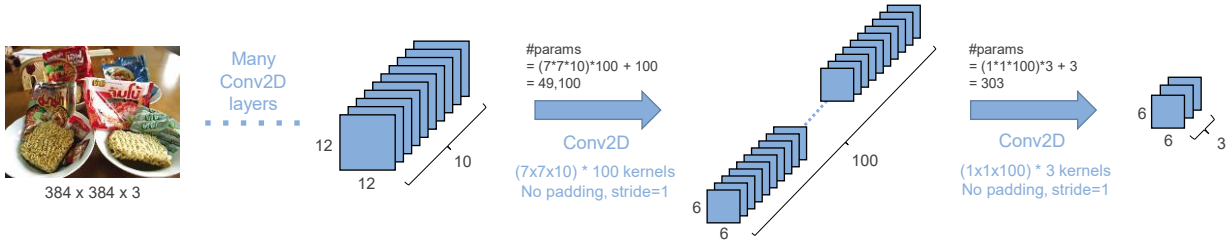


Further reading: http://cs231n.github.io/convolutional-networks/#convert

21

FC layer ⇔ Conv layer

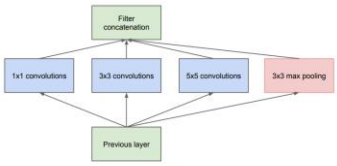




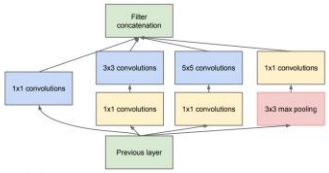
Further reading: http://cs231n.github.io/convolutional-networks/#convert

22

Inception v1 (ImageNet 2014, CVPR 2015)

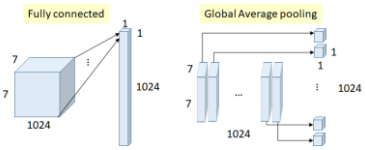


(a) Inception module, naïve version



(b) Inception module with dimension reductions

- **GoogLeNet (Inception v1)**: 22 layers
  - Winner of ImageNet 2014
  - Trained with Google's internal DL infrastructure, DistBelief (the distributed ML system)
- **Tricks:**
  - Inception module for parallel features
  - 1x1 convolution as a dimension reduction module to reduce computation
  - Multiple losses to ease the gradient problem
  - Used average pooling (no trainable parameter) instead of FC layers

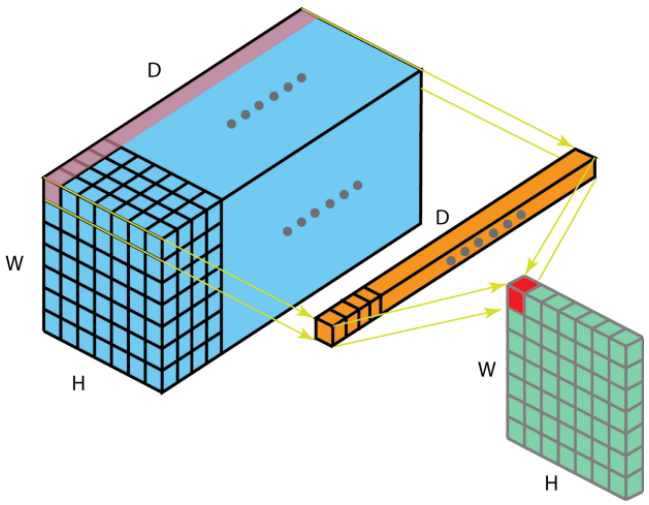


"Going deeper with convolution," in Procs. of IEEE CVPR, 2015 https://arxiv.org/abs/1409.4842

23

1x1 CONV2D

- Advantages:
  - Dimensionality reduction for efficient computations
  - Efficient low-dimensional embedding, or feature pooling
  - Applying nonlinearity again after convolution

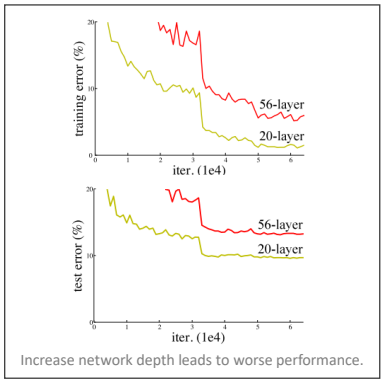


https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215

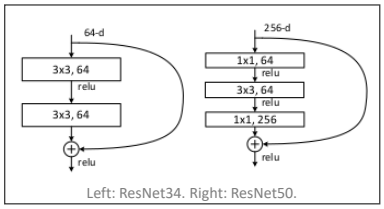
24

25

Microsoft ResNet (ImageNet 2015, CVPR 2016)

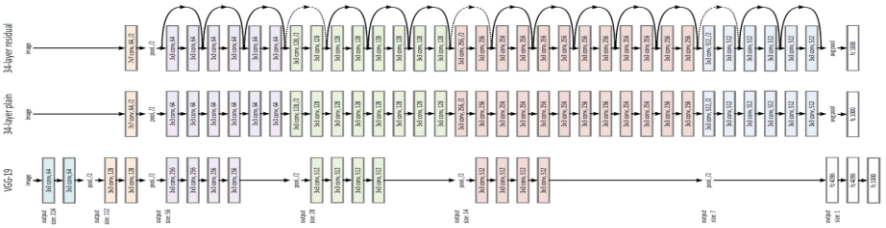


Increase network depth leads to worse performance.



Left: ResNet34. Right: ResNet50.

- **ResNet**: 152 layers
  - **ResNet-152** was the winner of ImageNet 2015
  - Finished 90-epoch ImageNet-1k training with **ResNet-50** on a NVIDIA M40 GPU takes 14 days
  - ResNet-50 originally performed with a minibatch size of 256 images (using 8 Tesla P100 GPUs, training time is 29 hours).
  - In November 2018, researchers from Sony (Japan) trained ResNet-50 (50 layers) on ImageNet dataset with the new record of 224 seconds. This was done using the GPU cluster with 1,088 nodes (each node contains four NVIDIA Tesla V100 GPUs). https://nnabla.org/paper/imagenet_in_224sec.pdf
- **Tricks:**
  - The core idea of ResNet is introducing a so-called “identity shortcut connection” that skips one or more layers.
    - Skip connections introduce shortcut paths for the gradient to flow through.
    - Ensure that the higher layer will perform at least as good as the lower layer, and not worse

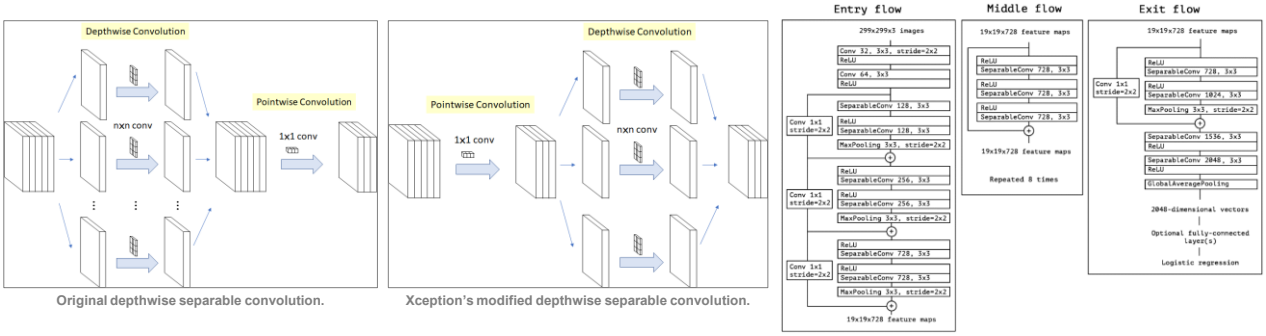


Microsoft Research, “Deep residual learning for image recognition,” In CVPR 2016, arXiv, 2015 https://arxiv.org/abs/1512.03385

26

Xception (CVPR 2017)

- **Xception** stands for Extreme version of Inception
  - Replace convolution in Inception with depthwise separable convolution to reduce computation
  - **Xception** is comparable with Inception-v3 (i.e., 1st runner up in ImageNet 2015) but is much faster.

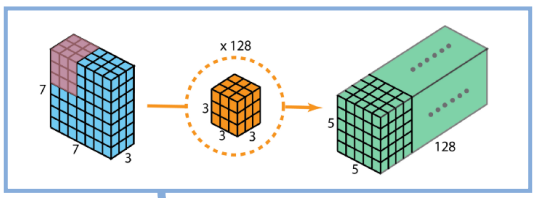


Francois Chollet, "Xception: Deep learning with depthwise separable convolutions," In Procs. of IEEE CVPR, 2017 https://arxiv.org/abs/1610.02357

27

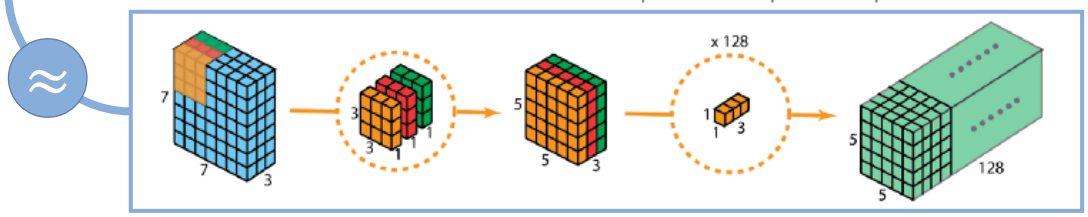
Depthwise Separable Convolution

- **PRO:** Much less operations for depthwise separable convolutions compared to 2D convolutions
- **CON:** Reduce the number of parameters in the convolution, so that, for a small model, the model capacity may be decreased significantly and may become sub-optimal.



Standard 2D convolution to create output with 128 layer, using 128 filters.

The overall process of depthwise separable convolution.

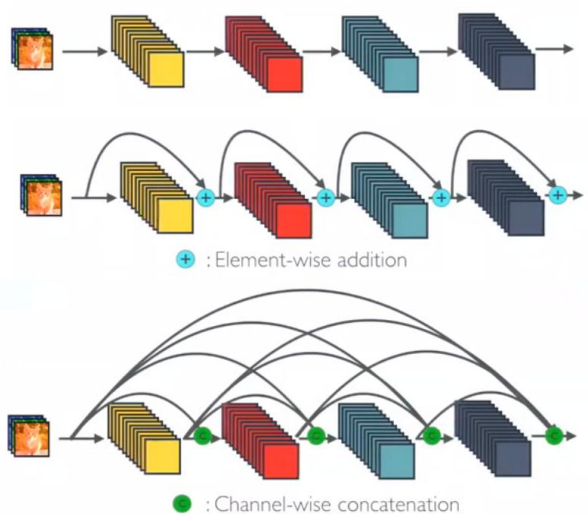


https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215

28

DenseNet (CVPR 2017)

- **DenseNet** (best paper @CVPR2017) from Cornell university, Tsinghua university and Facebook AI



Standard ConvNet concept:
In Standard ConvNet, input image goes through multiple convolution and obtain high-level features.

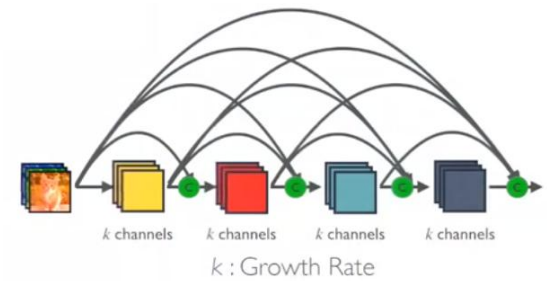
ResNet concept:
Identity mapping is proposed to promote the gradient propagation. Element-wise addition is used. It can be viewed as algorithms with a state passed from one ResNet module to another one.

One Dense Block in DenseNet:
Each layer obtains additional inputs from all preceding layers and passes on its own feature-maps to all subsequent layers. Concatenation is used. Each layer is receiving a "collective knowledge" from all preceding layers.

"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 https://arxiv.org/abs/1608.06993 | Image credit: 25NOV2018 https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

29

DenseNet (CVPR 2017)



Dense Block in DenseNet with growth rate k:
Since each layer receives feature maps from all preceding layers, network can be thinner and compact (i.e., number of channels can be fewer). The growth rate k is the additional number of channels for each layer.

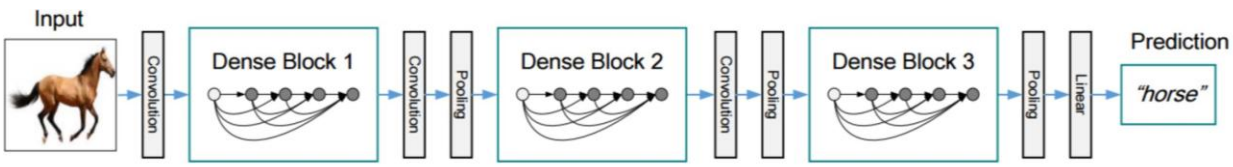


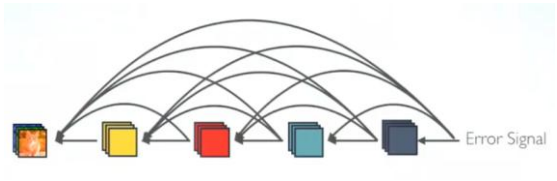
Figure 2. A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature map sizes via convolution and pooling.

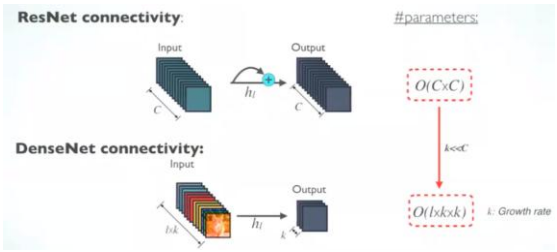
"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 https://arxiv.org/abs/1608.06993 | Image credit: 25NOV2018 https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

30

DenseNet (CVPR 2017)

- **PROs:**
  - $Strong\ gradient\ flow$: The error signal can be propagated to earlier layers more directly. This is a kind of implicit deep supervision as earlier layers can get direct supervision from the final classification layers.
  - $Parameter\ and\ computational\ efficiency$: For each layer, number of parameters in ResNet is directly proportional to $C \times C$ while number of parameters in **DenseNet** is directly proportional to $l \times k \times k$. Since $k << C$, **DenseNet** has much smaller size than ResNet.



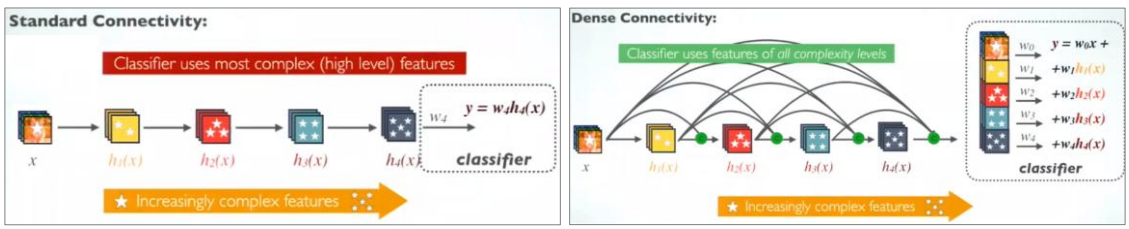


"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 https://arxiv.org/abs/1608.06993 | Image credit: 25NOV2018 https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

31

DenseNet (CVPR 2017)

- **PROs:**
  - $More\ diversified\ features$: Since each layer in **DenseNet** receive all preceding layers as input, more diversified features and tends to have richer patterns.
  - $Maintain\ low\ complexity\ features$:
    - In a standard ConvNet, classifier uses most complex features.
    - In **DenseNet**, classifier uses features of all complexity levels. It tends to give more smooth decision boundaries. It also explains why **DenseNet** performs well when training data is insufficient.



"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 https://arxiv.org/abs/1608.06993 | Image credit: 25NOV2018 https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

32

Google NASNet (CVPR 2018)

- **NASNet** stands for **N**eural **A**rchitecture **S**earch **Net**work
- Search for the best network configuration using reinforcement learning
- **NASNet** achieves low(er) computation while maintaining high accuracy.

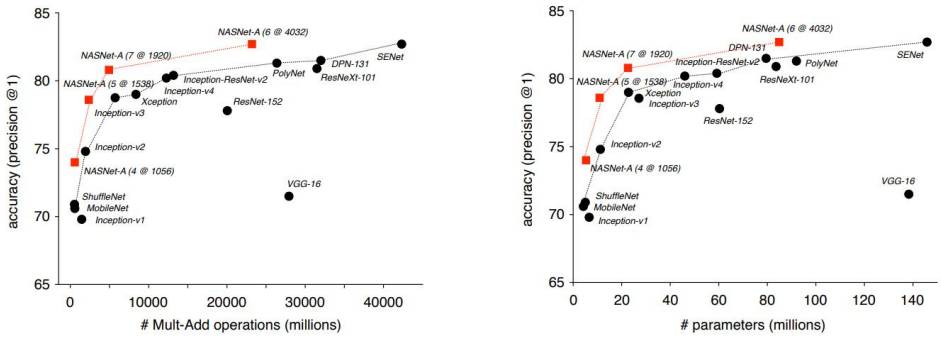


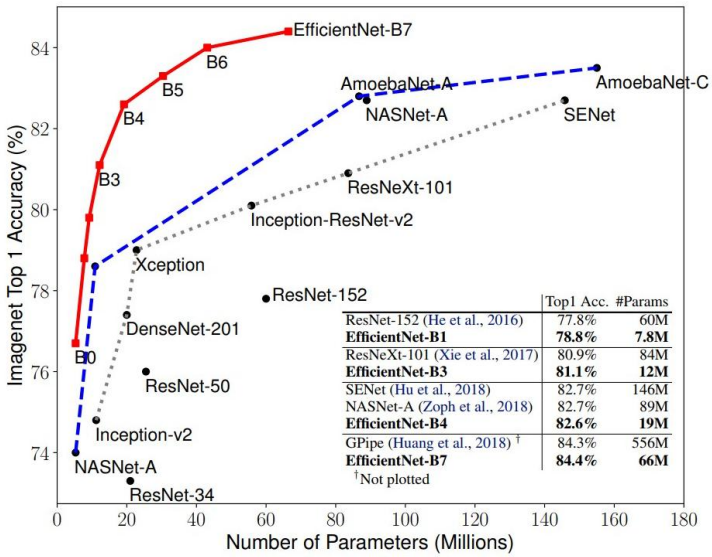
Figure 5. Accuracy versus computational demand (left) and number of parameters (right) across top performing published CNN architectures on ImageNet 2012 ILSVRC challenge prediction task. Computational demand is measured in the number of floating-point multiply-add operations to process a single image. Black circles indicate previously published results and red squares highlight our proposed models.

Google Research, "Learning Transferable Architectures for Scalable Image Recognition," In Procs. of IEEE/CVF CVPR, 2018 https://arxiv.org/abs/1707.07012

33

Google EfficientNet (ICML 2019)

- Google **EfficientNets** refer to a family of models which superpass state-of-the-art accuracy with up to 10x better efficiency (smaller and faster).



	Top1 Acc.	#Params
ResNet-152 (He et al., 2016)	77.8%	60M
EfficientNet-B1	**78.8%**	**7.8M**
ResNeXt-101 (Xie et al., 2017)	80.9%	84M
EfficientNet-B3	**81.1%**	**12M**
SENet (Hu et al., 2018)	82.7%	146M
NASNet-A (Zoph et al., 2018)	82.7%	89M
EfficientNet-B4	**82.6%**	**19M**
GPipe (Huang et al., 2018) †	84.3%	556M
EfficientNet-B7	**84.4%**	**66M**

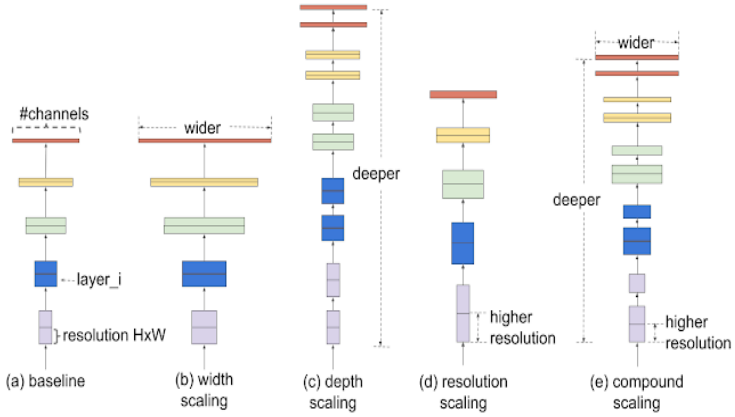
†Not plotted

Google Brain, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," In Procs. of ICML, 2019 https://arxiv.org/abs/1905.11946

34

Google EfficientNet (ICML 2019)

- Conventional model scaling:
  - The conventional practice is to arbitrarily increase the CNN depth or width, or to use larger input image resolution for training and evaluation.
  - While these methods do improve accuracy, they usually require tedious manual tuning, and still often yield suboptimal performance.
- Compound scaling method:
  - A novel model scaling method that uses a simple yet highly effective compound coefficient to scale up CNNs in a more structured manner.
  - Unlike conventional approaches that arbitrarily scale network dimensions, such as width, depth and resolution, this method uniformly scales each dimension with a fixed set of scaling coefficients.
  - This method consistently improves model accuracy and efficiency for scaling up existing models such as MobileNet (+1.4% imagenet accuracy), and ResNet (+0.7%), compared to conventional scaling methods.

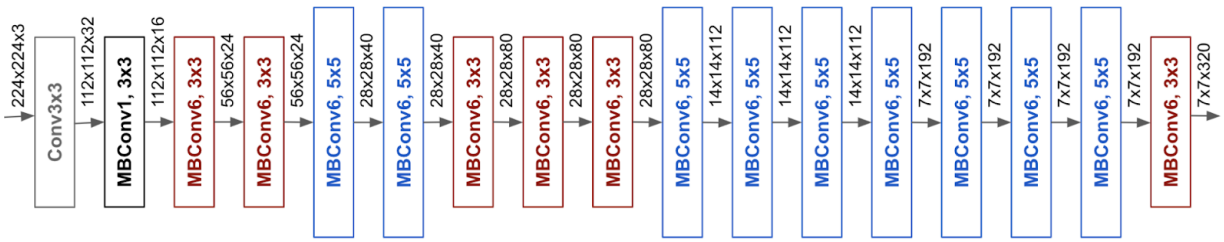


Google Brain, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," In Procs. of ICML, 2019 https://arxiv.org/abs/1905.11946

35

Google EfficientNet (ICML 2019)

- Baseline network:
  - The effectiveness of model scaling also relies heavily on the baseline network.
  - So, to further improve performance, a new baseline network is developed by performing a Neural Architecture Search (NAS) using the AutoML MNAS framework, which optimizes both accuracy and efficiency (FLOPS).
  - The resulting architecture uses mobile inverted bottleneck convolution (MBConv), similar to MobileNetV2 and MnasNet, but is slightly larger due to an increased FLOP budget.
  - A simple mobile-size baseline architecture developed by AutoML MNAS is called **EfficientNet-B0**, while **EfficientNet-B1** to **B7** are obtained by scaling up the baseline network.



EfficientNet-B0

Google Brain, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," In Procs. of ICML, 2019 https://arxiv.org/abs/1905.11946

36

CNN image classifiers in Keras

- Losses
- Data loading
- Built-in small datasets
- **Keras Applications**
- Mixed precision
- Utilities
- KerasTuner
- Code examples
- Why choose Keras?
- Community & governance
- Contributing to Keras
- KerasTuner

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7
NASNetMobile	23	74.4%	91.9%	5.3M	389	27.0	6.7
NASNetLarge	343	82.5%	96.0%	88.9M	533	344.5	20.0
EfficientNetB0	29	77.1%	93.3%	5.3M	132	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	186	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	186	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	210	140.0	8.8
EfficientNetB4	75	82.9%	96.4%	19.5M	258	308.3	15.1
EfficientNetB5	118	83.6%	96.7%	30.6M	312	579.2	25.3
EfficientNetB6	166	84.0%	96.8%	43.3M	360	958.1	40.4
EfficientNetB7	256	84.3%	97.0%	66.7M	438	1578.9	61.6
EfficientNetV2B0	29	78.7%	94.3%	7.2M	-	-	-
EfficientNetV2B1	34	79.8%	95.0%	8.2M	-	-	-
EfficientNetV2B2	42	80.5%	95.1%	10.2M	-	-	-
EfficientNetV2B3	59	82.0%	95.8%	14.5M	-	-	-
EfficientNetV2S	88	83.9%	96.7%	21.6M	-	-	-
EfficientNetV2M	220	85.3%	97.4%	54.4M	-	-	-
EfficientNetV2L	479	85.7%	97.5%	119.0M	-	-	-
ConvNeXtTiny	109.42	81.3%	-	28.6M	-	-	-
ConvNeXtSmall	192.29	82.3%	-	50.2M	-	-	-
ConvNeXtBase	338.58	85.3%	-	88.5M	-	-	-
ConvNeXtLarge	755.07	86.3%	-	197.7M	-	-	-
ConvNeXtXLarge	1310	86.7%	-	350.1M	-	-	-

The top-1 and top-5 accuracy refers to the model's performance on the ImageNet validation dataset.

Depth refers to the topological depth of the network. This includes activation layers, batch normalization layers etc.

Time per inference step is the average of 30 batches and 10 repetitions.

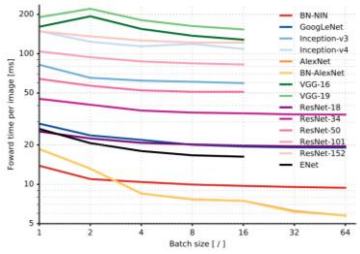
- CPU: AMD EPYC Processor (with IBPB) (92 core)
- RAM: 1.7T
- GPU: Tesla A100
- Batch size: 32

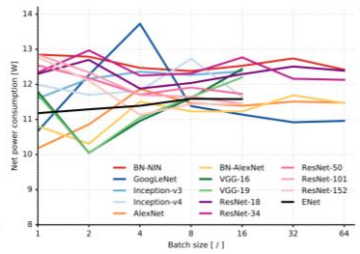
Depth counts the number of layers with parameters.

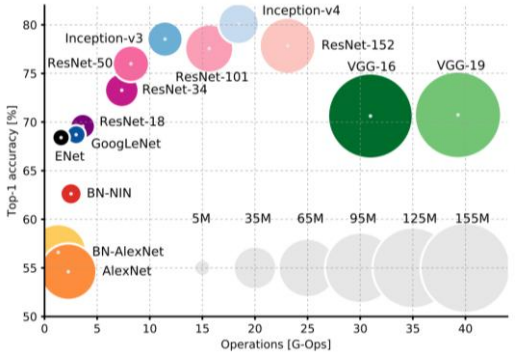
https://keras.io/applications/

37

Which CNN to choose?





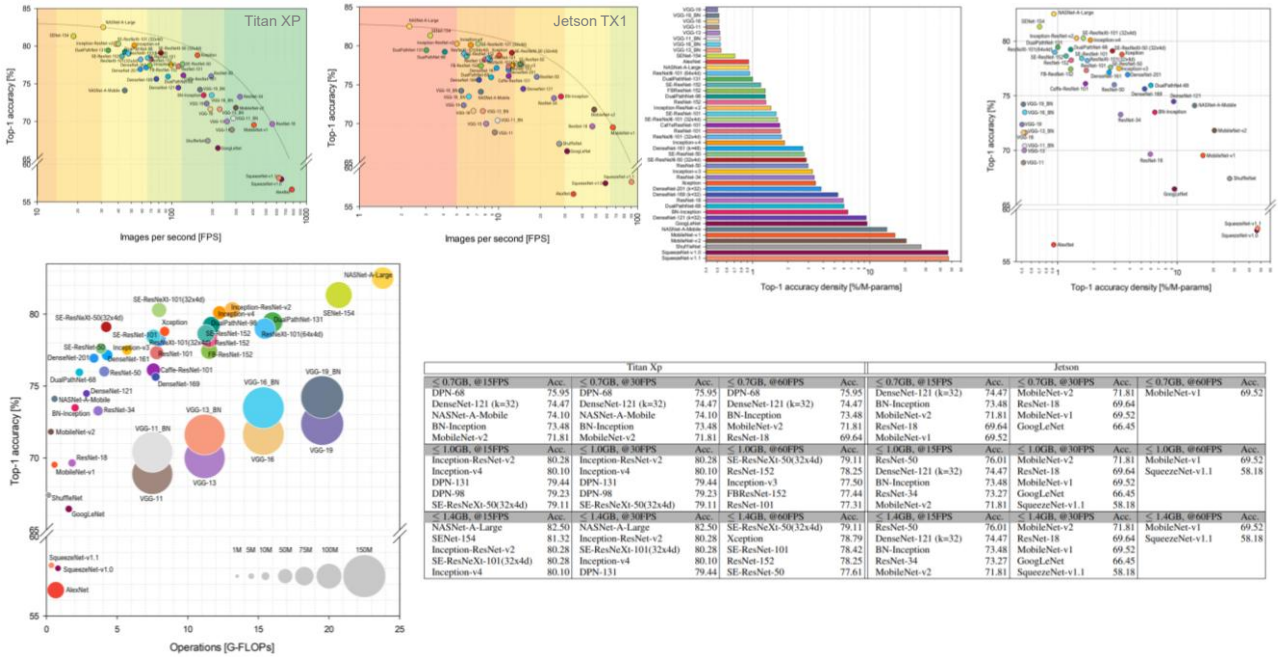


- **Accuracy:** how precise the model
- **FLOPs:** how many operations to perform
- **Model size:** how big is the model
- **RAM usage:** how much RAM the model consumes
- **Inference time:** how fast the model predicts an input
- **Power consumption:** how much power is used to run the model

- Note that small models do not automatically convey a small amount of RAM memory and FLOPs.

Canziani, Culurciello, and Paszke, "An Analysis of Deep Neural Network Models for Practical Applications," In arXiv, 2017: https://arxiv.org/abs/1605.07678

38



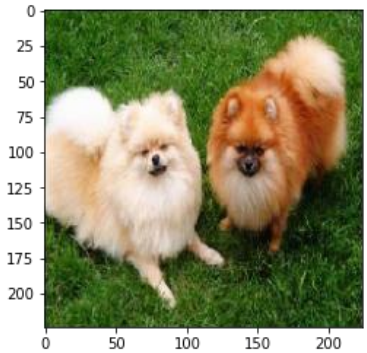
Titan Xp						Jetson					
< 0.7GB, @15FPS	Acc.	< 0.7GB, @30FPS	Acc.	< 0.7GB, @60FPS	Acc.	< 0.7GB, @15FPS	Acc.	< 0.7GB, @30FPS	Acc.	< 0.7GB, @60FPS	Acc.
DPN-68	75.95	DPN-68	75.95	DPN-68	75.95	DenseNet-121 (k=32)	74.47	MobileNet-v2	71.81	MobileNet-v1	69.52
DenseNet-121 (k=32)	74.47	DenseNet-121 (k=32)	74.47	DenseNet-121 (k=32)	74.47	BN-Inception	73.48	ResNet-18	69.64		
NASNet-A-Mobile	74.10	NASNet-A-Mobile	74.10	BN-Inception	73.48	MobileNet-v2	71.81	MobileNet-v1	69.52		
BN-Inception	73.48	BN-Inception	73.48	MobileNet-v2	71.81	ResNet-18	69.64	GoogLeNet	66.45		
MobileNet-v2	71.81	MobileNet-v2	71.81	ResNet-18	69.64	MobileNet-v1	69.52				
< 1.0GB, @15FPS	Acc.	< 1.0GB, @30FPS	Acc.	< 1.0GB, @60FPS	Acc.	< 1.0GB, @15FPS	Acc.	< 1.0GB, @30FPS	Acc.	< 1.0GB, @60FPS	Acc.
Inception-ResNet-v2	80.28	Inception-ResNet-v2	80.28	SE-ResNeXt-50(32x4d)	79.11	ResNet-50	76.01	MobileNet-v2	71.81	MobileNet-v1	69.52
Inception-v4	80.10	Inception-v4	80.10	ResNet-152	78.25	DenseNet-121 (k=32)	74.47	ResNet-18	69.64	SqueezeNet-v1.1	58.18
DPN-131	79.44	DPN-131	79.44	Inception-v3	77.50	BN-Inception	73.48	MobileNet-v1	69.52		
DPN-98	79.23	DPN-98	79.23	FBResNet-152	77.44	ResNet-34	73.27	GoogLeNet	66.45		
SE-ResNeXt-50(32x4d)	79.11	SE-ResNeXt-50(32x4d)	79.11	ResNet-101	77.31	MobileNet-v2	71.81	SqueezeNet-v1.1	58.18		
< 1.4GB, @15FPS	Acc.	< 1.4GB, @30FPS	Acc.	< 1.4GB, @60FPS	Acc.	< 1.4GB, @15FPS	Acc.	< 1.4GB, @30FPS	Acc.	< 1.4GB, @60FPS	Acc.
NASNet-A-Large	82.50	NASNet-A-Large	82.50	SE-ResNeXt-50(32x4d)	79.11	ResNet-50	76.01	MobileNet-v2	71.81	MobileNet-v1	69.52
SENet-154	81.32	Inception-ResNet-v2	80.28	Xception	78.79	DenseNet-121 (k=32)	74.47	ResNet-18	69.64	SqueezeNet-v1.1	58.18
Inception-ResNet-v2	80.28	SE-ResNeXt-101(32x4d)	80.28	SE-ResNet-101	78.42	BN-Inception	73.48	MobileNet-v1	69.52		
SE-ResNeXt-101(32x4d)	80.28	Inception-v4	80.10	ResNet-152	78.25	ResNet-34	73.27	GoogLeNet	66.45		
Inception-v4	80.10	DPN-131	79.44	SE-ResNet-50	77.61	MobileNet-v2	71.81	SqueezeNet-v1.1	58.18		

"Benchmark Analysis of Representative Deep Neural Network Architectures," In IEEE Access, 2018 https://arxiv.org/abs/1810.00736

39

EX2: The pre-trained image classifier

- Load a SOTA image classifier that is already pre-trained on ImageNet dataset

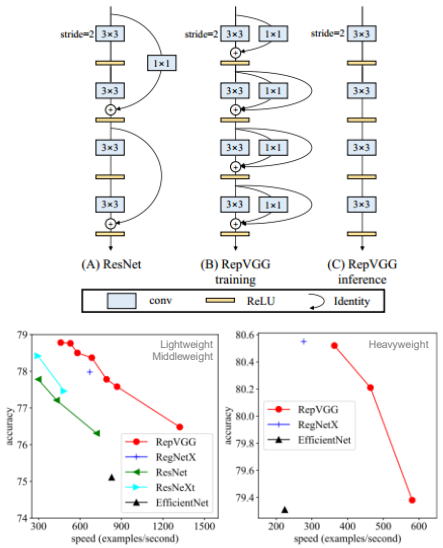


```
Rank 1: ('n02112018', 'Pomeranian', 0.9680393)
Rank 2: ('n02112137', 'chow', 0.016785022)
Rank 3: ('n02112350', 'keeshond', 0.0062348126)
Rank 4: ('n02086079', 'Pekinese', 0.005780382)
Rank 5: ('n02098413', 'Lhasa', 0.000932306)
Rank 6: ('n02085782', 'Japanese_spaniel', 0.00055715203)
Rank 7: ('n02097474', 'Tibetan_terrier', 0.00019835595)
Rank 8: ('n02096294', 'Australian_terrier', 0.00014964186)
Rank 9: ('n02086910', 'papillon', 0.00013190691)
Rank 10: ('n02094433', 'Yorkshire_terrier', 0.00011159651)
```

40

41

RepVGG (CVPR 2021)



Thitirat TheSensei
5 กุมภาพันธ์ 2021 ·

RepVGG: Make VGG great again

Intro: State-of-the-art VGG

พูดถึงชื่อ VGG ผลงานจากทีมวิจัย Oxford Visual Geometry Group เชื่อว่าคนสาย image/vision ส่วนใหญ่ต้องร้อง อ๋อ เพราะ VGG network คือ State-of-the-art (SOTA) CNN (Convolutional Neural Network) ตัวแรก ๆ ของยุค modern deep learning (DL) ที่โด่งดังจากการคว้าตำแหน่งรองชนะเลิศอันดับ 1 ในการแข่งขัน ImageNet 2014 (classification task)

จากปี 2014 วาร์ปกลับมาปัจจุบันปี 2021 ท่ามกลางโมเดล CNN รุ่นใหม่ที่เกิดขึ้นมากมาย ไม่น่าเชื่อว่า VGG ก็ยังคงสามารถยืนหยัดเป็นหนึ่งใน backbone ตัวเลือกยอดฮิตสำหรับสายงาน image/vision อยู่ได้ ... โดยจุดหนึ่งที่ VGG ดูแตกต่างจากคนอื่น (ในกลุ่มยอดฮิต) อย่างเห็นได้ชัด คือ ความเป็น network เพลน ๆ ง่าย ๆ ที่มีแค่ a stack of 3x3 convolution and ReLU เรียงกันตรงไปตรงมา ไม่มี skip connection ไม่มีทางแยก ไม่มี branch ไม่มีการแตกกิ่งก้านสาขาอะไรใน network ทั้งสิ้น

เรียกว่าถ้าใช้ DL framework ยุคปัจจุบันอย่าง TensorFlow หรือ PyTorch ... แม้แต่มือใหม่เพิ่งหัดเรียน CNN ก็ใช้ Sequential API เขียนโปรแกรมเลียนแบบ VGG architecture (from scratch) เสร็จได้ภายในไม่กี่นาที (ไม่นับเวลา train นะ) ... หรือถ้าอยากไม่ทางสายลัด ขอไว กว่านั้นอีก ก็เรียก pre-trained VGG ขึ้นมาใช้ได้จบ ใน DL framework มีเตรียมมาให้พร้อมใช้แล้ว

What's wrong with multi-branch designs?

ถ้าเทียบระหว่าง CNN backbones คุณลาสติกยอดฮิตอย่าง VGG vs. ResNet คำแนะนำที่มักบอกต่อ ๆ กัน คือ ถ้าคาดหวังผลลัพธ์ความแม่นยำที่ดีกว่า ควรเลือก ResNet ที่มี skip connection เนื่องจาก "a multi-branch architecture makes the model an implicit ensemble of numerous shallower models" ... ส่วน VGG นั้น ในแง่ความแม่นยำถือว่าเป็นรอง แต่มีข้อดีที่เป็นโครงสร้างลำดับขั้นเข้าใจง่าย เรียงจาก low-level features ค่อย ๆ ไป high-level features ไม่มีข้ามขั้น ซึ่งลักษณะนี้ก็เหมาะสมกับงานบางอย่างโดยเฉพาะ (เช่น การทำ style transfer)

- https://www.facebook.com/thitirat.thelecturer/posts/2824859631067610

"RepVGG: Making VGG-style ConvNets Great Again," In Procs. of CVPR, 2021 https://arxiv.org/abs/2101.03697

42

DeepMind NFNets (ICLR 2021)

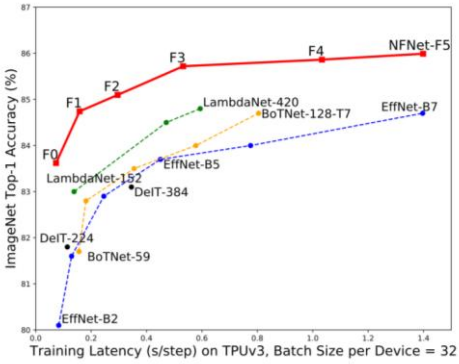


Figure 1. **ImageNet Validation Accuracy vs Training Latency.** All numbers are single-model, single crop. Our NFNet-F1 model achieves comparable accuracy to an EffNet-B7 while being 8.7× faster to train. Our NFNet-F5 model has similar training latency to EffNet-B7, but achieves a state-of-the-art 86.0% top-1 accuracy on ImageNet. We further improve on this using Sharpness Aware Minimization (Foret et al., 2021) to achieve 86.5% top-1 accuracy.

➢ https://www.facebook.com/thitirat.thelecturer/posts/2831757527044487

Google Deepmind, "High-Performance Large-Scale Image Recognition Without Normalization," In arXiv, 2021 https://arxiv.org/abs/2102.06171
Google Deepmind, "Characterizing signal propagation to close the performance gap in unnormalized ResNets," In Procs. of ICLR, 2021 https://arxiv.org/abs/2101.08692

43

44

Foundation model

Jump start our CNNs with the big foundation models and their pre-trained weights

45

THE MODERN HISTORY

- **2006:** The pretraining technique by Hinton et al.

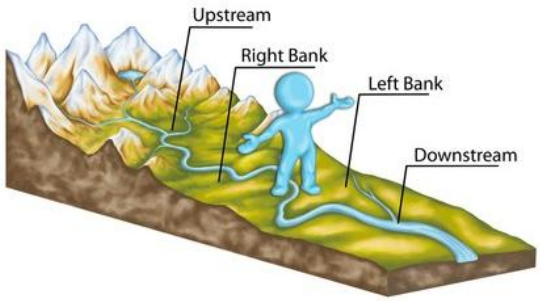
- **2012:** ImageNet evolution—AlexNet 
- **2014:** Birth of GRU, VAE, and GAN
- **2015:** Birth of diffusion model
- **2017:** The DeepFake viral
- **2017:** Birth of the groundbreaking Transformer

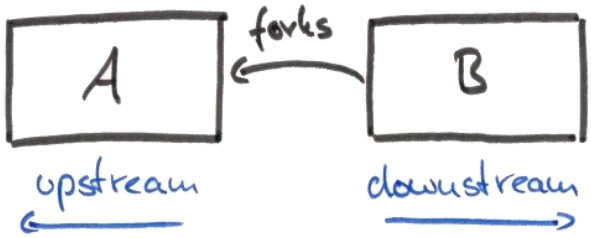
- **2018:** NLP's ImageNet moment—BERT
- **2018:** ACM Turing Award to Deep NN

46

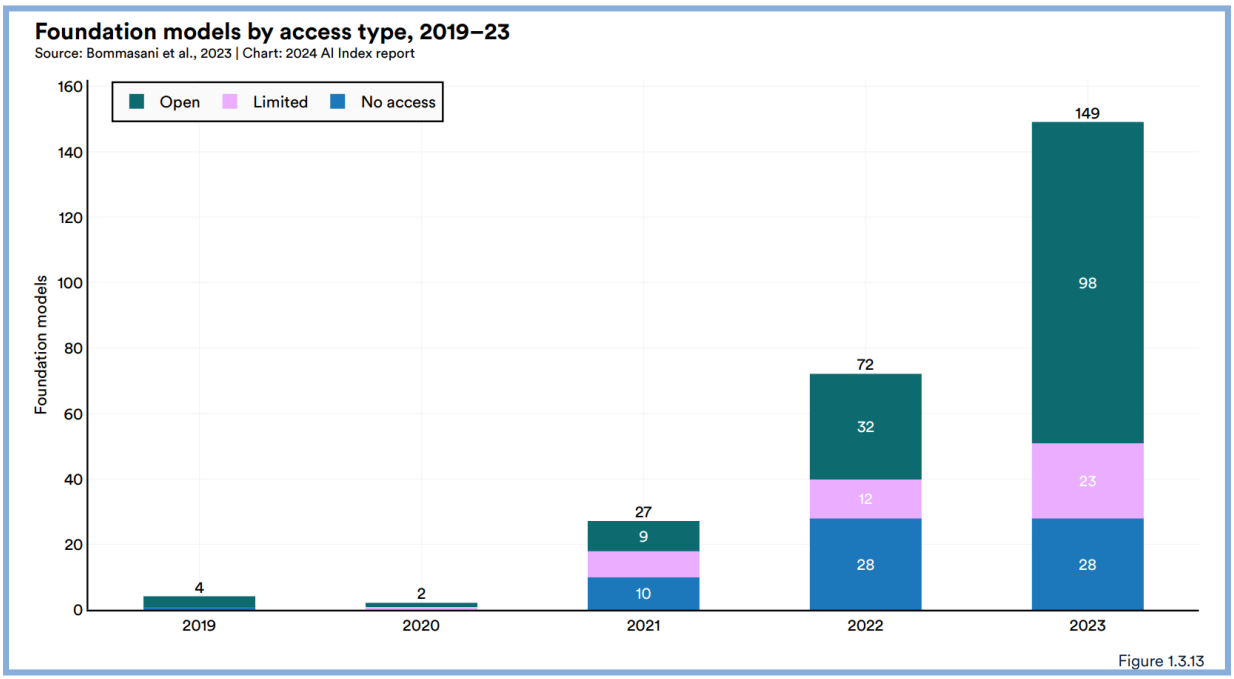
Downstream / Upstream tasks

- **Downstream task:**
  - This is what the deep learning field calls those supervised-learning tasks that utilize a pre-trained model or component. http://jalammar.github.io/illustrated-bert/
- **Upstream task:**
  - In the context of open source projects, *A* is the upstream project since it can very well live without project B. But project B (the fork, the downstream project) wouldn't even exist without project *A* (the original project). https://reflectoring.io/upstream-downstream/



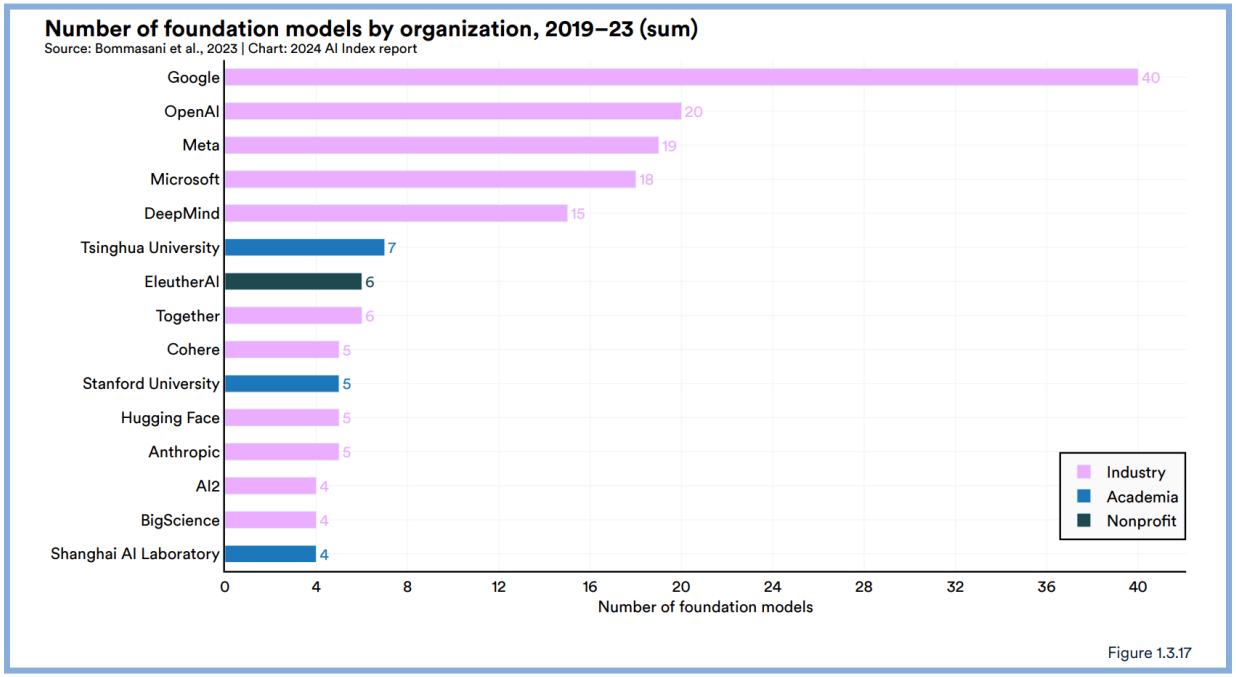


47



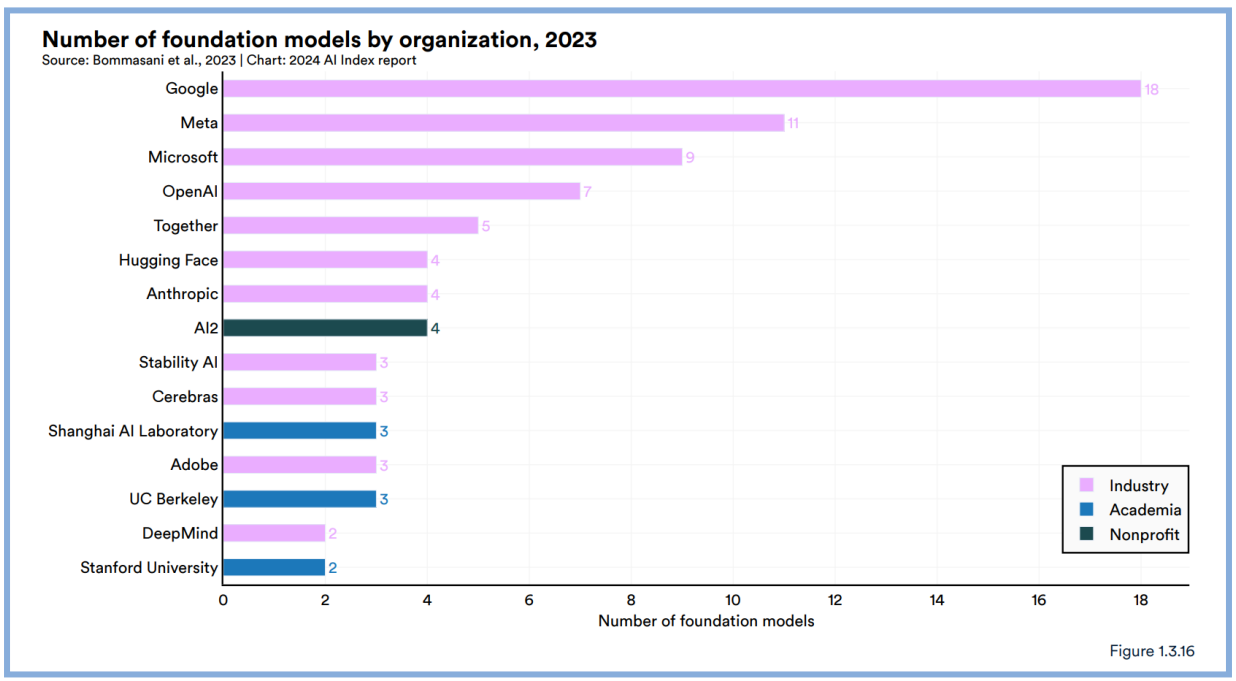
HAI AI Index Report 2024 (April 2024), https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI_AI-Index-Report-2024.pdf

48



HAI AI Index Report 2024 (April 2024), https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI_AI-Index-Report-2024.pdf

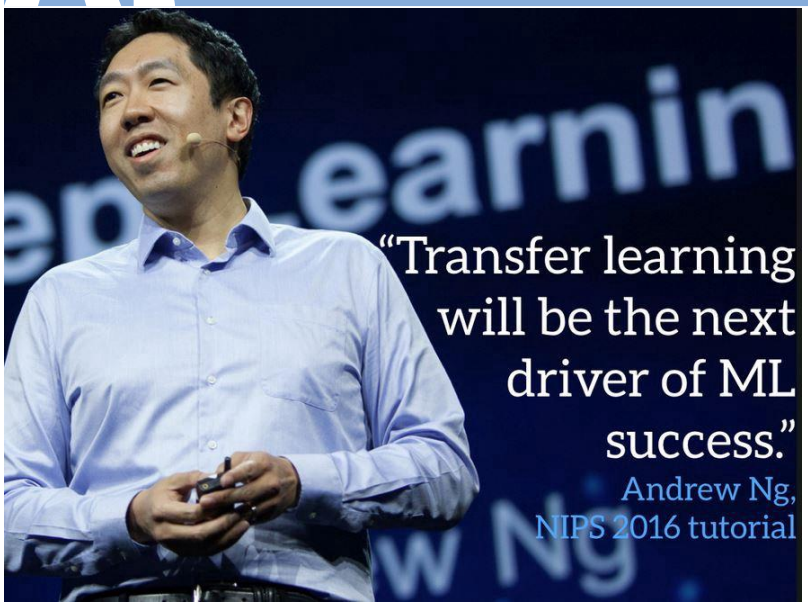
49

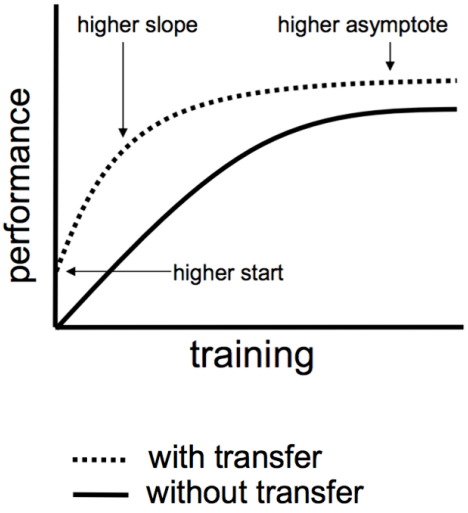


HAI AI Index Report 2024 (April 2024), https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI_AI-Index-Report-2024.pdf

50

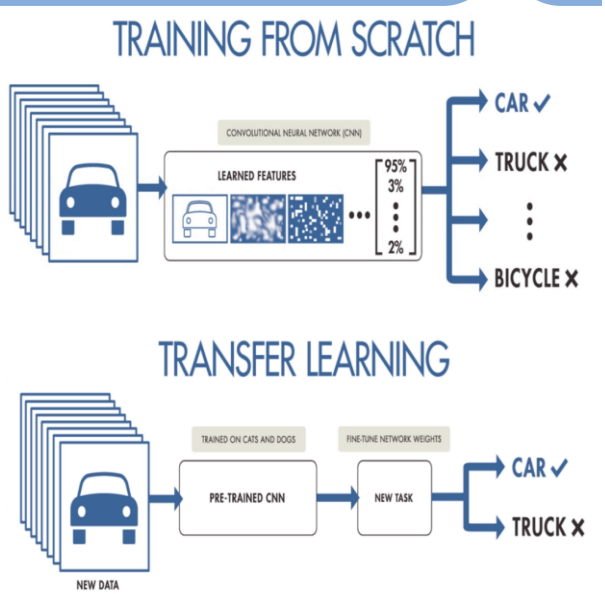
Transfer Learning





51

Transfer Learning



52

Finetuning

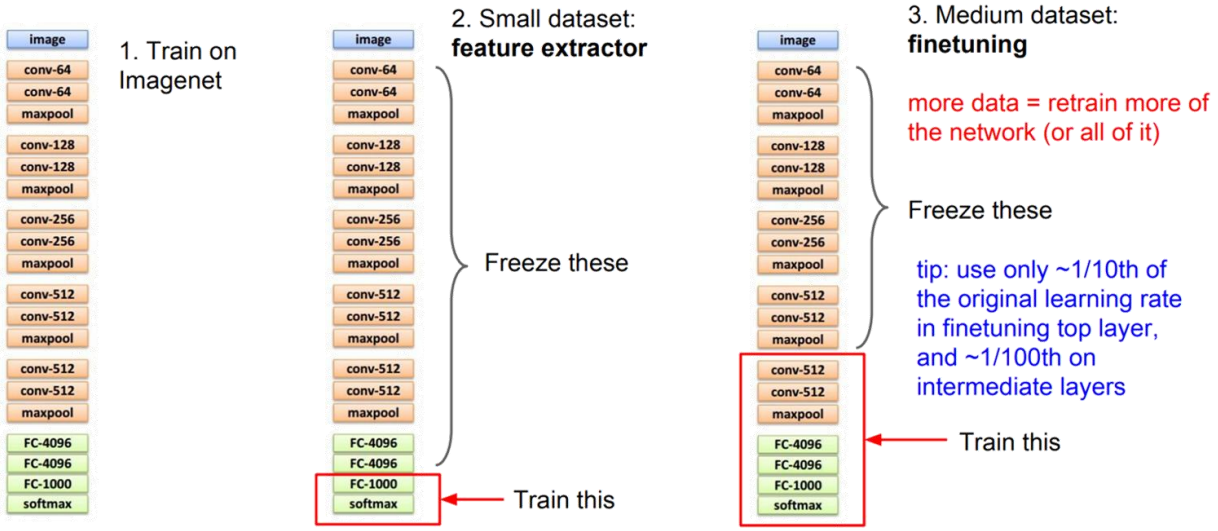


Image credit: http://cs231n.stanford.edu/slides/2016/winter1516_lecture11.pdf

53

Visual Feature hierarchy

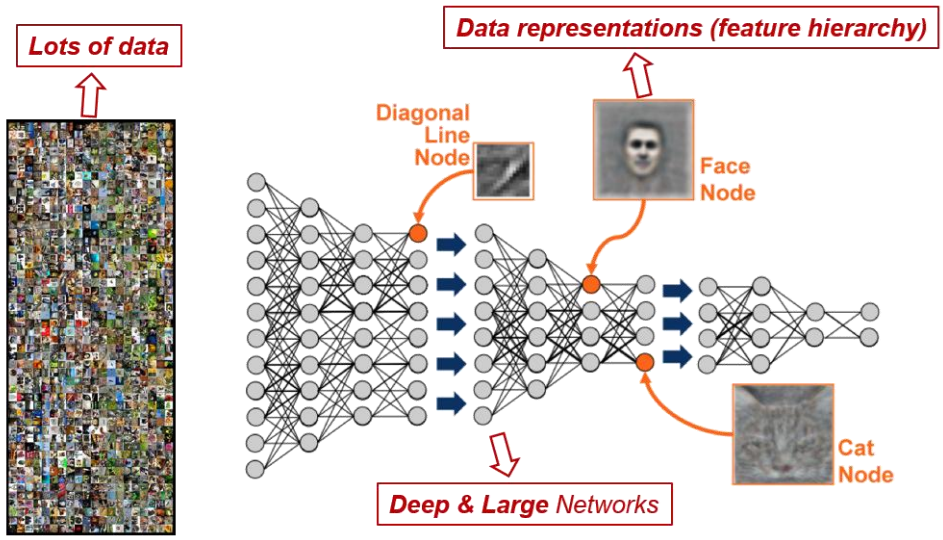
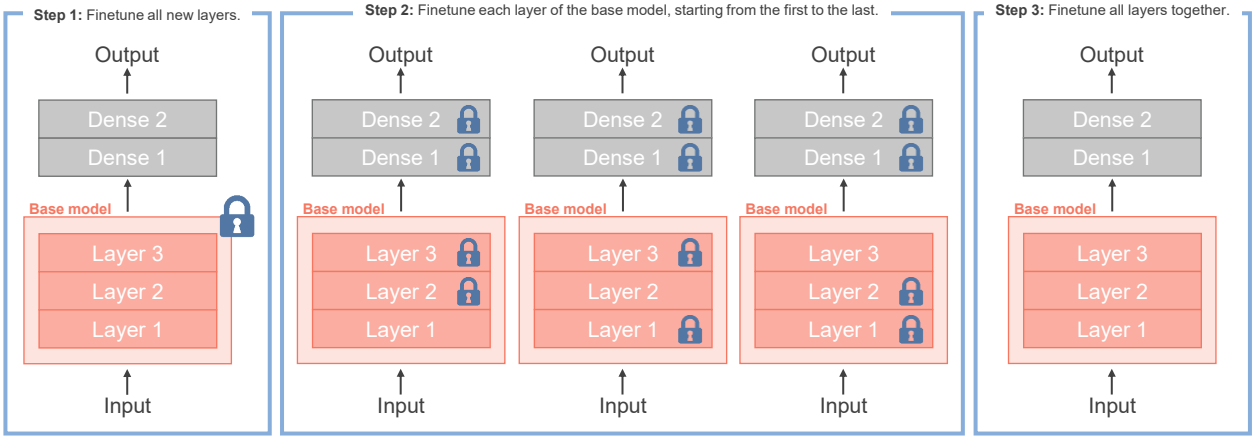


Image credit: http://brain.kaist.ac.kr/research.html

54

Chain-Thaw Finetuning (2017)

- A transfer learning approach that sequentially unfreezes and finetunes a single layer at a time
- Increase accuracy on the target task at the expense of extra computational power needed for the finetuning

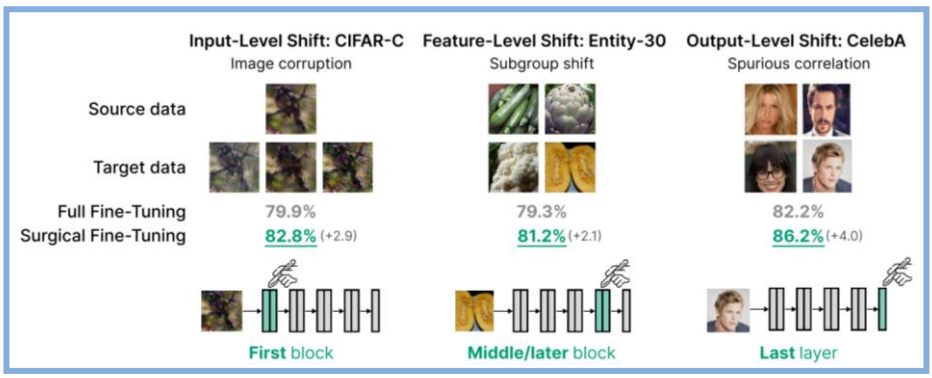


Felbo et al., "Using millions of emoji occurrences to learn any-domain representations for detecting sentiment, emotion and sarcasm," In Procs. of EMNLP 2017 https://arxiv.org/abs/1708.00524

55

Surgical Finetuning (ICLR 2023)

- A common approach to transfer learning under distribution shift is to finetune the last few layers of a pre-trained model. This is to fit the new data while also preserving the information obtained during the pre-training phase.
- **Surgical finetuning** shows that selectively finetuning a subset of layers matches or outperforms commonly used finetuning approaches.

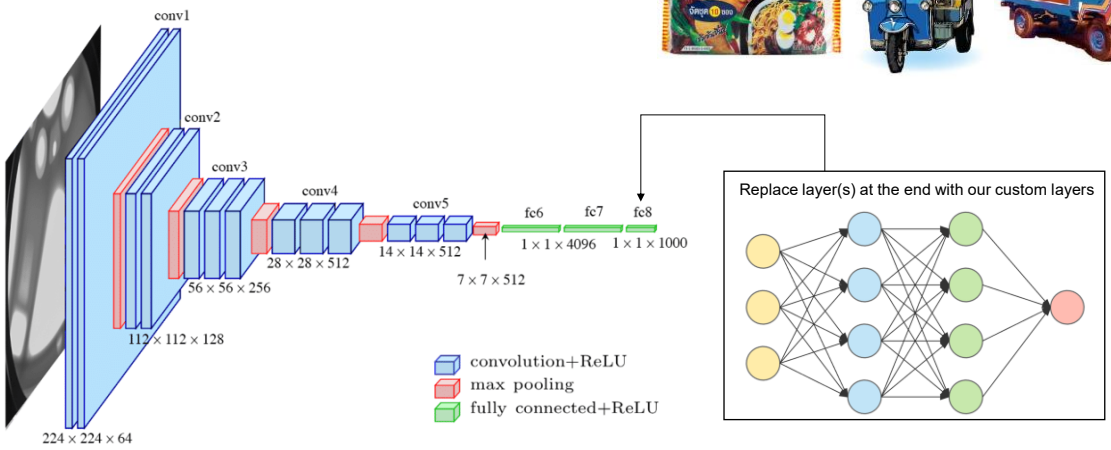


Stanford university, "Surgical Fine-Tuning Improves Adaptation to Distribution Shifts," arXiv 2022, ICLR 2023
https://arxiv.org/abs/2210.11466

56

EX3: Downstream image classifier

- Different domain (e.g., ImageNet vs. CIFAR-10) but same task (image classification)



57

Be Careful …

- How big is our dataset? How similar our dataset compared to the original dataset?

	Small	Large
Similar	Train only the (last) FC layer.	Freeze the earlier layers (simple features) and train the rest of layers.
Different	Train only the FC layers.	Train the model from scratch and reuse the network architecture (using the trained weights as a started point).

- Should we re-train the existing layer(s) or add the new layer(s)?
- What architecture to be used in the newly added layers?
- One or multiple transfer learnings?

Thitirat TheSensei
27 มกราคม เวลา 13:39 น. ·

#MetaAI ประยุกต์ใช้ AI มาสร้าง animation จากภาพวาดตัวการ์ตูนในจินตนาการของเด็ก ๆ

.

เมื่อช่วงกลางเดือนธันวาคม 2021 เห็นคุณ Mark Zuckerberg แห่ง Meta แชร์เว็บ https://sketch.metademolab.com/canvas นี้มา ... ลองเข้าไปกดเล่นกับตัว sample figure แล้วก็น่ารักดีนะ ดูตัวการ์ตูนลายเส้นเด่นกันกระดุ๊กกระดิ๊ก

.

พอได้อ่านลงรายละเอียดการพัฒนาใน blog แล้ว ก็เจอประเด็นน่าสนใจหลายประเด็นที่ไม่ใช่การเสนอ state-of-the-art (SOTA) ตัวใหม่ แต่เป็นการชี้ให้เห็นถึงจุดอ่อนของพวก pretrained vision models (deep learning: DL) ใน task ปกติที่คนชอบทำกันอย่าง object detection และ segmentation ที่สุดท้ายก็ต้องวนกลับไปที่ classical techniques ที่ใช้ human knowledge นี่ละเข้าไปช่วยอุดรูโหว่ของปัญหาไปพลาง ๆ (จนกว่าจะมี training data มากและหลากหลายครอบคลุมพอ) ... ว่าแล้วเลยมาเขียนสรุปเอาไว้สักหน่อย

.

****

.

Why kid's drawings are difficult?

.

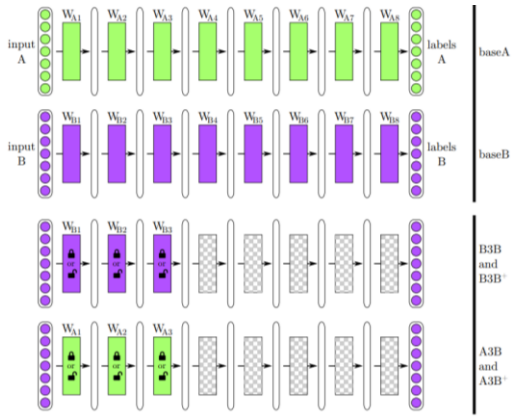
ในปัจจุบันด้วยอานิสงค์ของ SOTA DL model หลายตัวที่ถูก pretrain มาให้อย่างดีแล้ว การจะทำ object detection หรือ segmentation บนภาพ คน สัตว์ หรือ figure อะไรสักอย่างเลยดูจะไม่ใช่เรื่องยาก

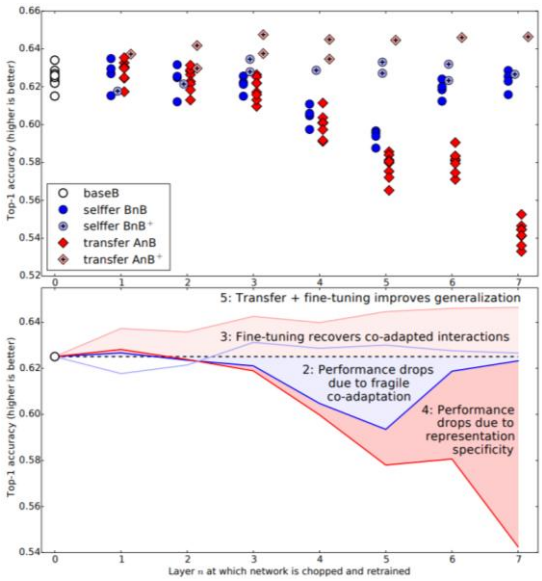
https://www.facebook.com/thitirat.thelecturer/posts/3096122700607967

58

Co-adaptation issue

- When neurons on neighboring layers co-adapt during training in such a way that cannot be rediscovered when one layer is frozen





Yosinki et al., "How transferable are features in deep neural networks?," NIPS 2014 https://arxiv.org/abs/1411.1792

59

Catastrophic Forgetting / Negative Transfer

- **Catastrophic Forgetting:**
  - Achieving artificial general intelligence requires that intelligent agents must demonstrate a capacity for continual learning—the ability to learn in a sequential fashion, accumulating the knowledge learnt in previous tasks, and using it to help future learning.
  - Continual learning is challenging for neural networks as neural networks tend to abruptly lose previously learnt knowledge (of task A) as information relevant to the current task (task B) is incorporated. This phenomenon is called **Catastrophic Forgetting**.
  - Continue reading: "Overcoming catastrophic forgetting in neural networks," arXiv 2DEC2016, https://arxiv.org/abs/1612.00796.

- **Negative Transfer:**
  - When labelled data is scarce for a specific target task, transfer learning often offers an effective solution by utilizing data from a related source task.
  - However, when transferring knowledge from a less related source, it may inversely hurt the target performance, a phenomenon known as **Negative Transfer**.
  - Continue reading: "Characterizing and avoiding negative transfer," CVPR2019, https://ieeexplore.ieee.org/document/8953565.

60

Assess transferability of a pre-trained model

- **Linear probing:**
  - Attach a linear classifier to the pre-trained model's feature extractor
  - Then, train the model (while freezing the feature extractor) on a task-specific dataset and evaluate the performance
  - This method is simple and straightforward but is limited to linear classifiers and may not capture complex relationships.
- **Finetuning:**
  - Unfreeze some or all of the pre-trained model's layers and train to update their weights using task-specific data
- Many more ideas can be found in the latest publications that propose training a new foundation model.

61

Demystify CNN

Make CNNs more transparent and visually explainable

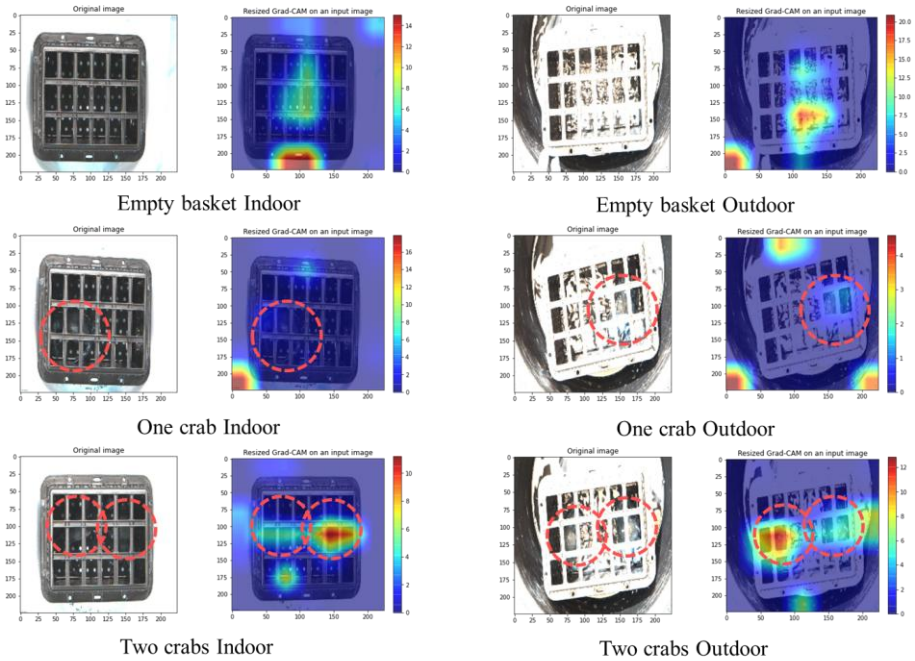
62

Why debugging CNN?

- How can we trust the decisions of a model if we cannot properly validate how it arrived there?
  - Where the network is looking in the input image?
  - Which series of neurons activated in the forward pass during inference/prediction?
  - How the network arrived at its final output?
  - …

- **For example:**
  - We may accidentally create a CNN model that classifies human genders by looking at the background color instead of looking at a human's face.
  - However, this CNN may yield 100% accuracy on both train and test sets.

63

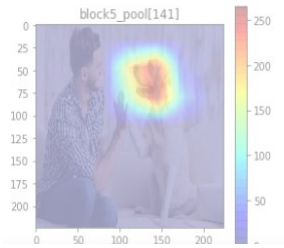


W. Siripattanadilok and T. Siriborvornratanakul, "Recognition of Partially Occluded Soft-shell Mud Crabs Using Deep Learning-based Image Classification and Object Detection," Aquaculture International, 2024

64

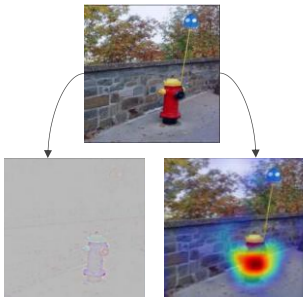
Visualizing what CNN learns

INTERMEDIATE ACTIVATIONS



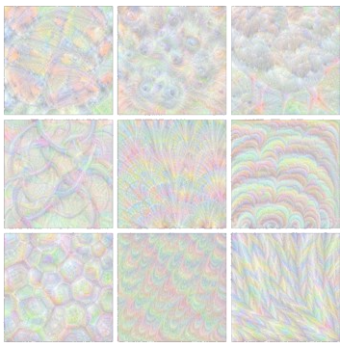
- Useful for understanding how successive convolutional layers transform their input
- Useful for getting a first idea of the meaning of individual convolutional filters

WHERE EACH CLASS LOOKS



- Useful for understanding which parts of an image were identified as belonging to a given class, allowing us to localize objects in images

CNN FILTERS



- Useful for understanding precisely what visual pattern or concept each filter in a CNN is receptive to

65

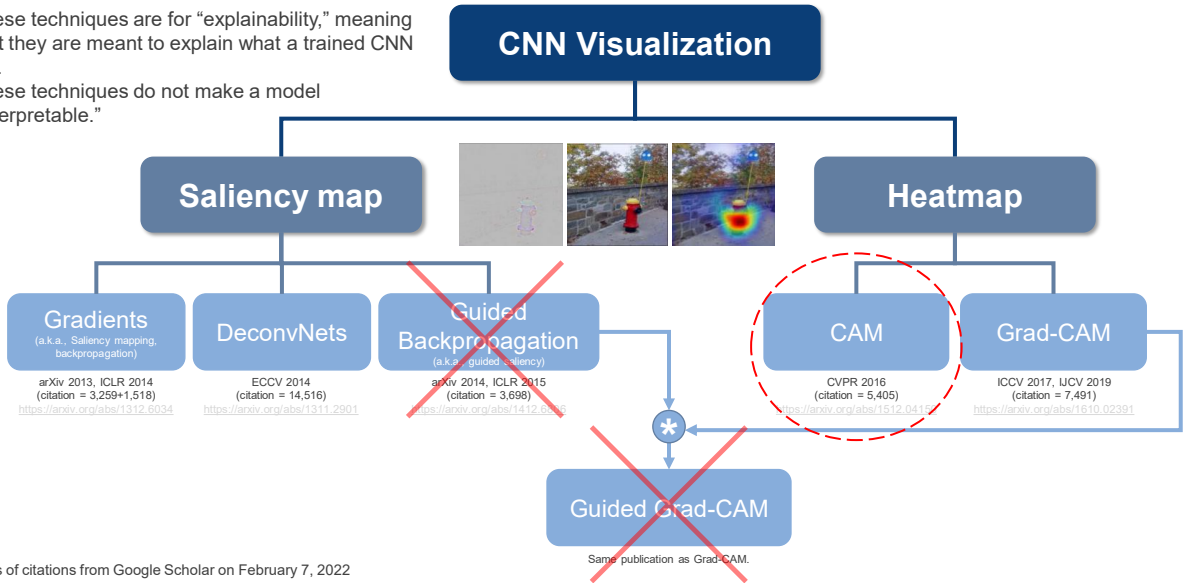
Blackbox AI vs. eXplainable AI

- Explainability means that it's possible to "explain" how a model made its decision.
  - **BUT** the explanation is not guaranteed to make sense to humans.
  - **AND** the explanation is not constrained to follow any known rules of the natural world.
  - **AND** the explanation is not guaranteed to be fair or free from biases.
  - For example:
    - A model may "explain" a boat classification by highlighting the water.
    - A model may "explain" a severely ill classification by highlighting a label within a medical image that indicates that the image was taken while the patient was lying down.
- Interpretability means that a model has been designed from the beginning to produce a human-understandable relationship between the inputs and the outputs.
  - For example:
    - Logistic regression is an interpretable model, in which the design of the model results in weights that show which inputs contribute more or less to the final predictions.
    - Rule-based methods are also interpretable.
- Explainability is not the same as interpretability!!!

66

Visualizing where each class looks

- These techniques are for "explainability," meaning that they are meant to explain what a trained CNN did.
- These techniques do not make a model "interpretable."

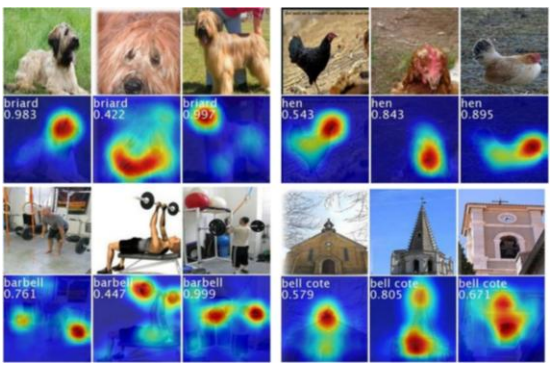


All numbers of citations from Google Scholar on February 7, 2022

67

Class Activation Map (CAM)

- **Class Activation Map (CAM)** was introduced in the paper entitled "Learning Deep Features for Discriminative Localization." (MIT @CVPR2016) https://arxiv.org/abs/1512.04150
  - The output of **CAM** is "a class-discriminative localization map," or a heatmap where the hot part corresponds to a discriminative region used by CNN to identify the output class. If there are n possible output classes, then for one input image, there can be n different **CAM** heatmaps as well.
  - This technique allows the classification-trained CNN to classify the image (as usual) and localize class-specific image regions in a single forward pass.



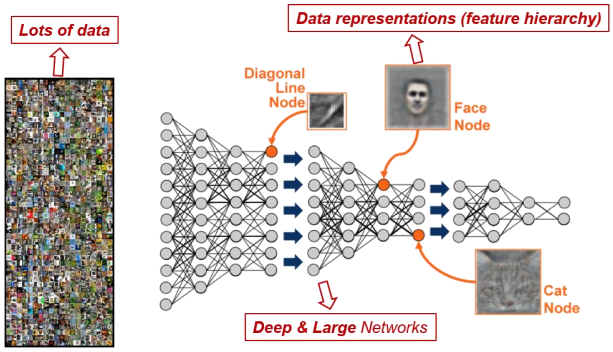
68

Class Activation Map (CAM)

- "*Think of a deep network as a multistage information-distillation operation, where information goes through successive filters and comes out increasingly purified.*"

  – Francois Chollet, 2018.

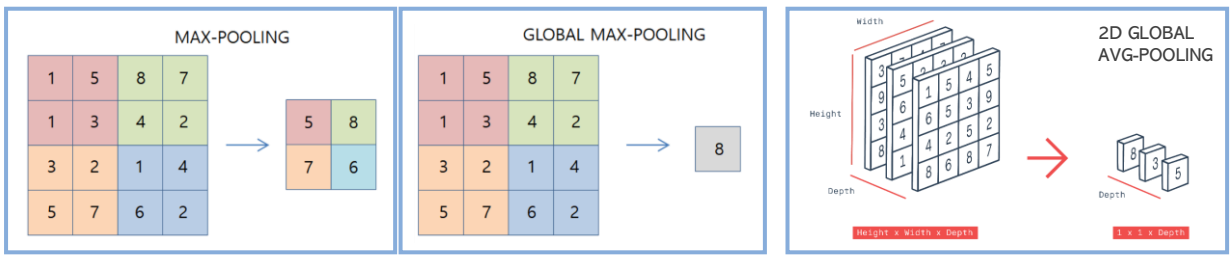
- Several previous works have asserted that deeper representations in a CNN capture the best high-level constructs.
- Also, CNNs naturally retain spatial information which is lost in fully-connected layers.
- Hence, we can expect the last convolutional layer to have the best tradeoff between high-level semantics and detailed spatial information.



69

Global Pooling Layer

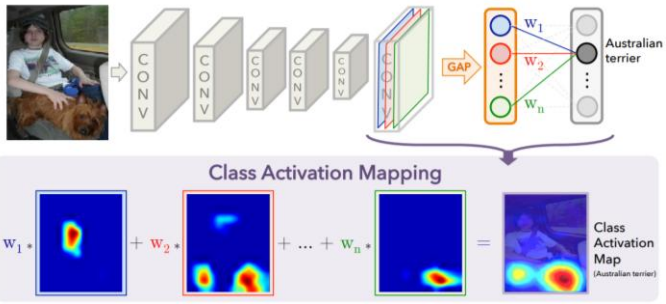
- Apart from Average Pooling and Max Pooling, there is another type of pooling called Global Pooling.
- Instead of down sampling patches of the input feature map, global pooling down samples the entire feature map to a single value. This would be the same as setting the `pool_size` to the size of the input feature map.
- Global pooling can be used in a model to aggressively summarize the presence of a feature in an image.
- Global pooling is also sometimes used in models as an alternative to using a FC layer to transition from feature maps to an output prediction for the model. This is to minimize overfitting by reducing the total number of parameters in the model.
- Both global average pooling and global max pooling are supported by Keras.



70

Class Activation Map (CAM)

- **CAM** is simply a weighted linear sum of the presence of visual patterns at different spatial locations.
  - Global Average Pooling (GAP) is preferred over Global Max Pooling layer (GMP) here. Because in GAP, we take an average across all the activation which helps to find all discriminative regions (whereas GMP layer just considers only the most discriminative one).
  - We can identify the importance of the image regions by projecting back the weights of the output layer on the convolutional feature maps obtained from the last convolutional layer.

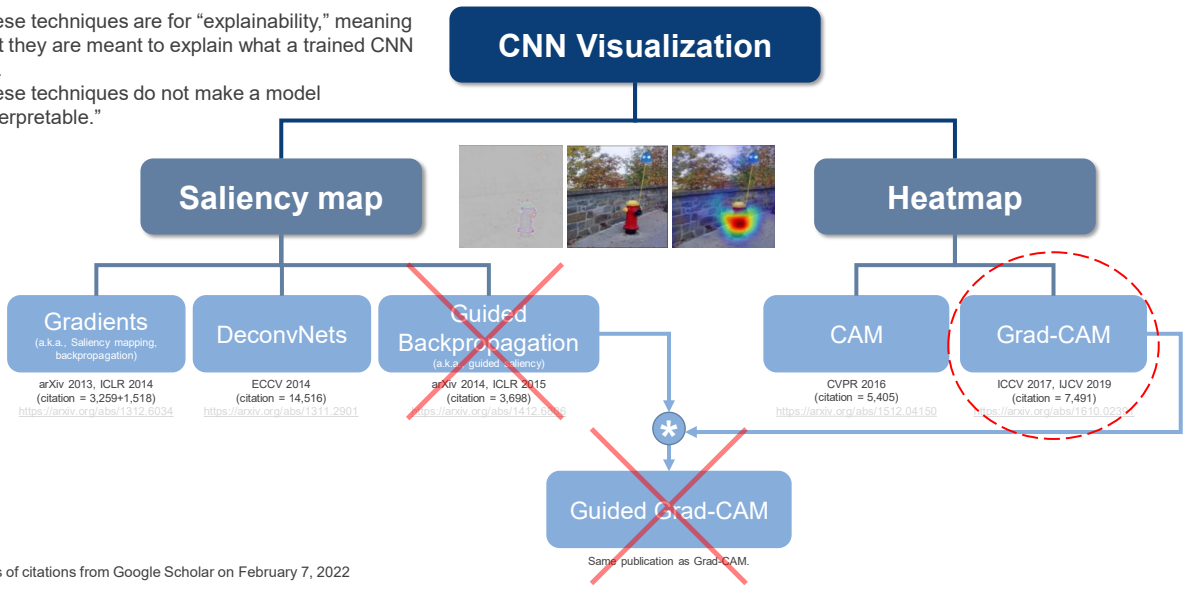


- Note that the same technique can be used for regression problems and other losses.

71

Visualizing where each class looks

- These techniques are for "explainability," meaning that they are meant to explain what a trained CNN did.
- These techniques do not make a model "interpretable."



All numbers of citations from Google Scholar on February 7, 2022

72

Gradient-weighted CAM (Grad-CAM)

- Limitations of CAM:
  - CAM requires feature maps to directly precede the softmax layers. Hence, it is only applicable to a particular kind of CNN architectures that perform global average pooling over convolutional maps immediately before prediction (i.e., conv feature maps > global average pooling > softmax layer).
  - This kind of restricted architecture may achieve in inferior accuracies compared to general networks on some tasks or simply be inapplicable to new tasks.

- **Gradient-weighted Class Activation Map (Grad-CAM)**(nickname: a gradient camera) was introduced in the paper entitled "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." (Georgia Institute of Technology @ICCV2017 @IJCV2019) https://arxiv.org/abs/1610.02391

- **Grad-CAM** can be seen as a generalization of CAM. In fully-convolutional architecture, **Grad-CAM** reduces to CAM.

73

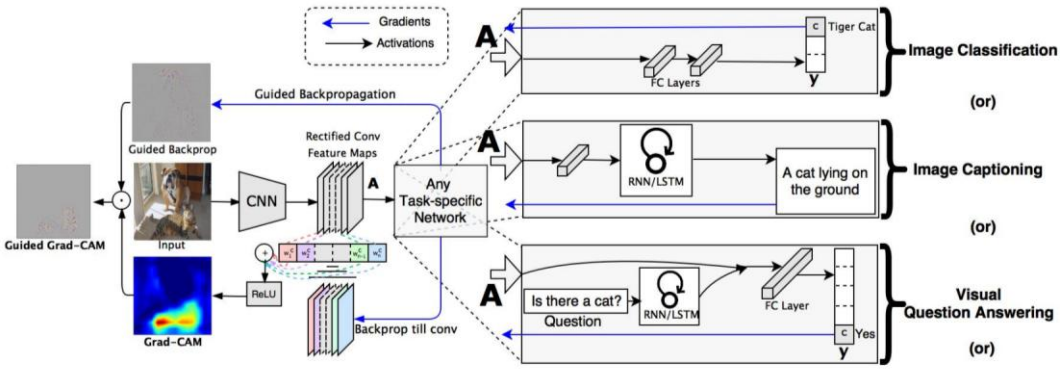
Gradient-weighted CAM (Grad-CAM)

- CAM vs. **Grad-CAM**:
  - Like CAM, the output of **Grad-CAM** is a class-discriminative localization map. So it can be used to visually debug where a network is looking in an image.
  - Like CAM, **Grad-CAM** uses the feature maps produced by the last convolutional layer of a CNN.
  - Unlike CAM, **Grad-CAM** is easily implemented, requires no modifications to the existing model architecture and needs no specific architecture.
    - In **Grad-CAM**, there can be any architecture following the last convolutional layer.
    - **Grad-CAM** works with nearly any CNN-based architecture, including those for image captioning and visual question answering.
  - Unlike CAM, **Grad-CAM** involves gradient computation. Hence, the only requirement in **Grad-CAM** is that layers following the last convolutional layer have to be differentiable so that gradients can be computed.

74

Gradient-weighted CAM (Grad-CAM)

- How **Grad-CAM** works:
  - In our explanation, we assume the image classification task where:
    - y_c corresponds to the raw class score output (i.e., the unnormalized predicted value before Softmax) of the network regarding the target class c.
    - Suppose that there are k 2D feature maps produced after the last convolutional layer, then A_i refers to the i^{th} feature map activation (height=v, width=u) when $i \in \{1, 2, 3, \dots, k\}$.



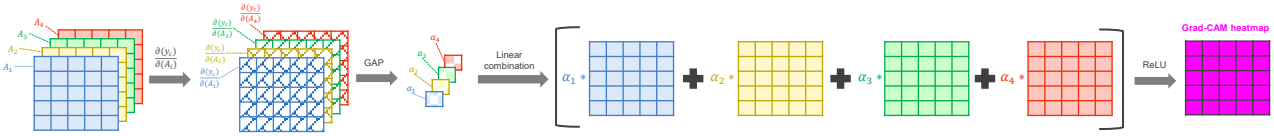
75

Gradient-weighted CAM (Grad-CAM)

- How **Grad-CAM** works:
  1. Compute the gradient of y_c with respect to each feature map activation A_i; this means computing $\frac{\partial(y_c)}{\partial(A_i)}$ for all $i \in \{1, 2, 3, \dots, k\}$
     - For a 2D input image, the resultant gradient of this step has a 3D shape of (k, v, u).
     - The resultant gradient can be seen as a stack of k maps where each map is a 2D gradient (shape=(v, u)) of y_c with respect to a single 2D feature map A_i.
  2. Compute a vector of k alpha values or $\{\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_k\}$, where α_i is a scalar obtained by global average pooling the 2D gradient (computed regarding A_i).
  3. Compute the final Grad-CAM heatmap (shape=(v, u)) by

$$Heatmap = ReLU(\alpha_1 A_1 + \alpha_2 A_2 + \dots + \alpha_k A_k)$$

ReLU is applied to emphasize only the positive gradient values and turn all the negative gradient values into zero.



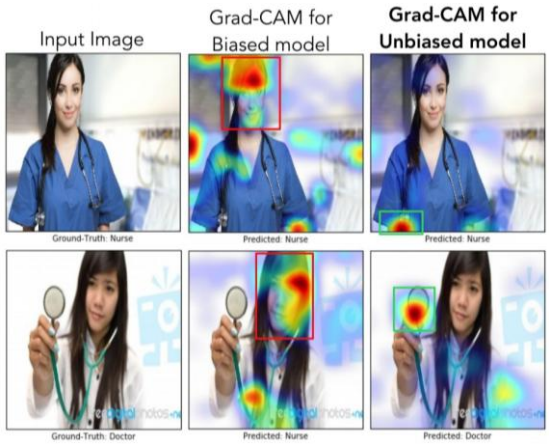
76

Gradient-weighted CAM (Grad-CAM)

- **Grad-CAM** is a technique for “explainability”—it is meant to explain what a trained CNN did.
- **Grad-CAM** does not make a model “interpretable”.

- While **Grad-CAM** heapmaps often make sense, they aren’t required to make sense.

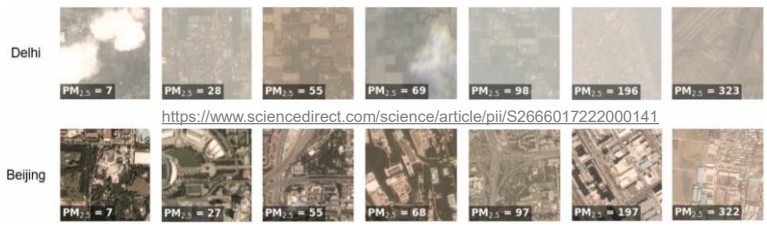
- **Grad-CAM** is useful for:
  - Weakly-supervised localization or weakly supervised segmentation
  - Debugging a model and gaining more insight into why the model is making certain mistakes



77

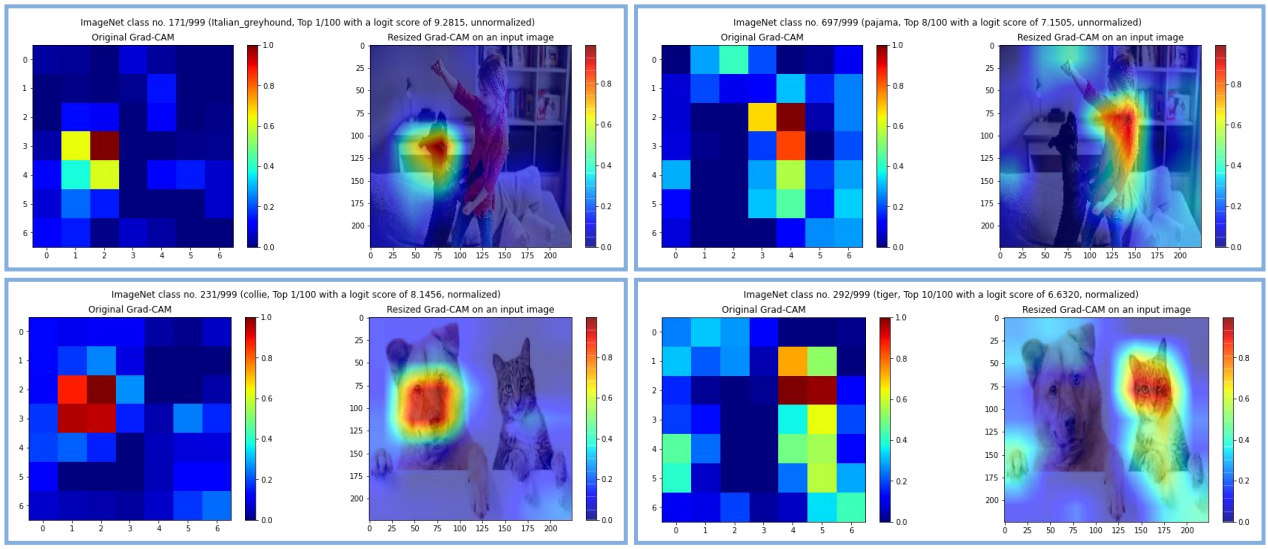
Gradient-weighted CAM (Grad-CAM)

- **Grad-CAM** IS NOT quite useful when:
  - **Multiple Instances of Same Class:** Difficult to differentiate individual objects when many of the same class are present.
  - **Small or Fine-Grained Objects:** Coarse resolution of heatmaps limits detailed explanation for tiny or intricate features.
  - **Spurious Correlations:** May highlight irrelevant background elements the model falsely associated with the class.
  - **Noisy or Heavily Occluded Images:** Unreliable results due to compromised gradient signals.
  - **Pixel-Level Granularity Needed:** Not ideal for explaining at the exact pixel level.
  - **Global Context Decisions:** Less effective when the model relies on overall image context, not just local features.
  - **Precise Multi-Class Localization:** Challenges in clearly distinguishing heatmaps for closely interacting multiple classes.



78

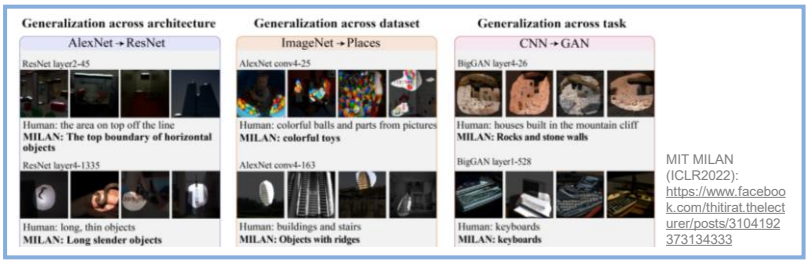
EX4: Visualizing CNN's Grad-CAM



79

Recent advances in XAI

Google StylEx (ICCV2021): https://www.facebook.com/thitirat.thelecturer/posts/3094986647388239



MIT MILAN (ICLR2022): https://www.facebook.com/thitirat.thelecturer/posts/3104192373134333

80

Homework

81

Image: Extend the pre-trained CNN

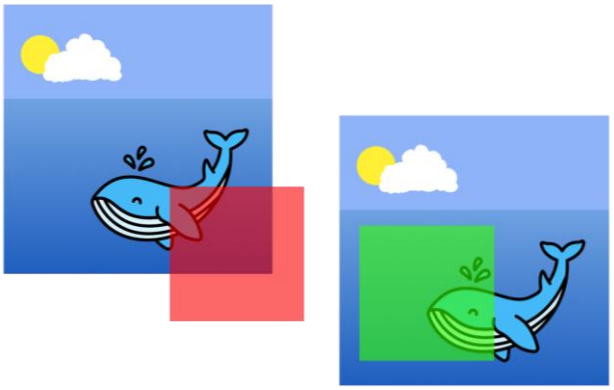
- **Objective:** What does it take to train an image classifier with our custom image dataset?
- Collect your own set of images (no duplication between groups) that cannot be classified or detected properly with models pre-trained on ImageNet
  - Either take images yourselves or collect them from various sources on the Internet
  - **DO NOT** use an existing dataset that someone else has already collected it
  - At submission, one group should contribute one new image dataset to their Github. If the images are collected from many sources, don't forget to give credit to all original contributors.
  - Be careful not to violate any law at this stage, both domestically and internationally !!!
- Select at least 3 pre-trained models, and use them as the backbones of your models
  - For an image classifier, it can be trained as either single-label or multi-label classification, depending on the dataset
  - Compare the performances of your models regarding each backbone
  - Compare performances of your models, on the ImageNet dataset versus those on your dataset
  - Be careful to use an appropriate set of evaluation metrics for your selected task
  - Make sure that your numeric performances are the true ones, not the tricked one
- Compare the original pre-trained CNN models (without your finetuning) and your models
  - Compare performances of the original pre-trained CNN, on the ImageNet dataset versus those on your dataset

Note that if your output classes do not exist in the original pre-trained model, just observe how good/bad the original model performs on your images and conclude it.

82

Unmatched image dimension, what to do?

Resize



Random crop

RandomResizedCrop
https://pytorch.org/vision/main/generated/torchvision.transforms.RandomResizedCrop.html

83

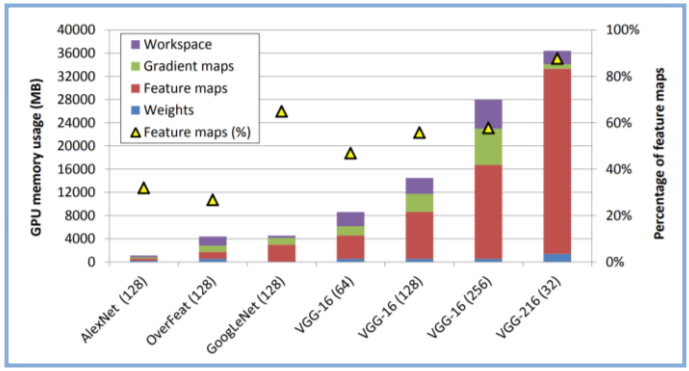
MEMORY REQUIREMENT

- During inference, the GPU RAM occupied by one layer can be released as soon as the next layer has been computed. Hence, we only need as much GPU RAM as required by two consecutive layers.
- **BUT** during training, everything computed during the forward pass needs to be preserved for the reverse pass, so the amount of GPU RAM needed is (at least) the total amount of RAM required by all layers.

84

Anatomy of Model's Memory

- Apart from the model itself, there are many components during training that use GPU memory. (Ref: https://huggingface.co/docs/transformers/perf_train_gpu_one)
  - Model weights
  - Optimizer states
  - Gradients
  - Forward activations saved for gradient computation
  - Temporary buffers
  - Functionality-specific memory
- For CNN, feature maps consume the most GPU memory.



NVIDIA, "vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design," MICRO, 2016. https://arxiv.org/abs/1602.08124

85

OUT-OF-MEMORY ERROR

- Use another GPU with bigger memory
- Train on multiple GPUs (for example, model parallelism, data parallelism)

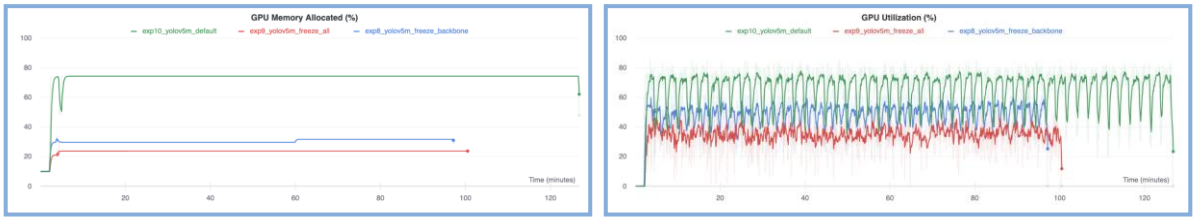
- Use smaller data (for example, smaller batch size, smaller image size)
- Use a smaller model (for example, remove some layers)
- Use a model architecture that consumes less GPU memory (for example, use pooling or strided convolution to reduce the dimension of intermediate feature maps)

- Reduce memory usage for parameters and computed values (for example, use the FP16 training mode or the mixed-precision training mode)
- Don't load the whole training dataset to GPU RAM (for example, use `model.train_on_batch()` instead of `model.fit()`, write a custom training loop)

86

OUT-OF-MEMORY ERROR

- Does freezing some layers help lower GPU memory usage?



Credit: https://docs.ultralytics.com/yolov5/tutorials/transfer_learning_with_frozen_layers/#gpu-utilization-comparison

- Other techniques to reduce GPU memory when training on a single GPU:
  (Ref: https://huggingface.co/docs/transformers/perf_train_gpu_one)

Method	Improve speed	Improve memory
Gradient accumulation	No	Yes
Gradient checkpointing	No	Yes
Mixed precision training	Yes	(No)
Batch size	Yes	Yes
Optimizer choice	Yes	Yes
DataLoader	Yes	No
DeepSpeed Zero	No	Yes

87

Ablation Study

- **An ablation study** comes where certain parts of the network are removed in order to gain a better understanding of the network's behavior.

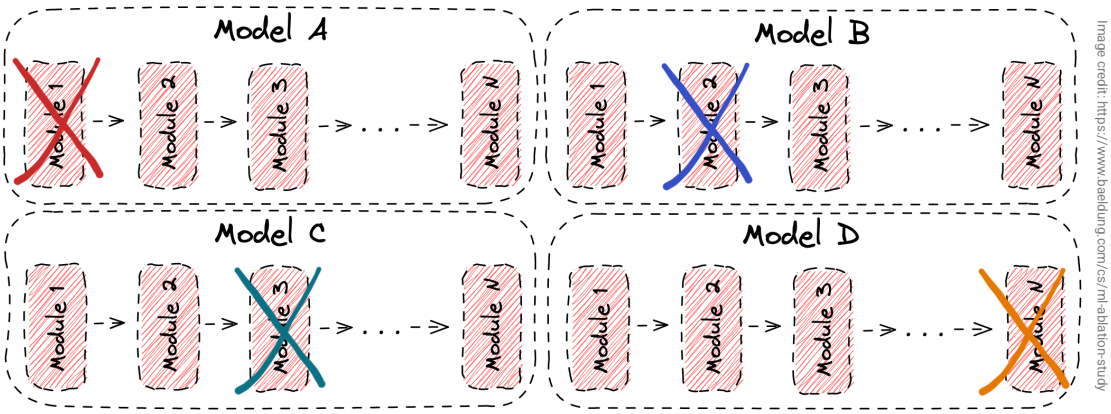


Image credit: https://www.baeldung.com/cs/ml-ablation-study

Each time we remove one of the modules and check the performance of the new model to investigate the influence of the removed module.

88

89